CoreTrace - Análise exploratória v2

Contents

Objetivo . . . 1
Contexto do cenário . . . 2
Preparação e leitura dos dados . . . 2
Organização dos servidores e das observações . . . 3
Qualidade dos dados . . . 4
Definição das métricas . . . 5
Origem dos dados e método . . . 9
Estado dos servidores no instante final . . . 11
Incidentes: início, escalada, máximo e recuperação . . . 12
Filtros, cards e barras da visão geral . . . 14
Evolução dos servidores em alerta . . . 20
Investigação técnica dos recursos . . . 25
Previsão de capacidade: geral e detalhe . . . 35
Justificativa das métricas essenciais . . . 38
Como a dashboard apoia as decisões . . . 39
Conclusão . . . 40
Referências e origem dos dados . . . 40
Exportação dos resultados . . . 40

Objetivo

Nesta análise, observamos o uso dos recursos de dez servidores em uma simulação de delivery. Queremos entender quando aparecem alertas, como eles mudam durante a semana e quais informações ajudam a investigar cada situação.

Também buscamos explicar a escolha das métricas do CoreTrace e como os indicadores e gráficos podem ajudar quem acompanha os servidores.

Os dados são simulados. Por isso, as conclusões valem para este cenário. Para saber se um alerta afetou pedidos ou pagamentos, também seriam necessários dados da aplicação.

Contexto do cenário

O cenário acompanha sete dias de uma operação de delivery, entre 12 e 19 de outubro de 2026. O último horário é 20h30 do dia 19, durante o aumento de uso dos recursos no jantar. Há um registro por minuto para cada servidor.

Os dez servidores estão divididos em três perfis, de acordo com a função de cada serviço:

- **Carrinho:** quatro servidores responsáveis pelos serviços de montagem e finalização de pedidos.
- **Financeiro:** três servidores responsáveis pelos serviços de pagamentos e transações.
- **Catálogo:** três servidores responsáveis por consultas e disponibilização de dados de produtos.

Durante a semana simulada, alguns servidores apresentam aumento no uso de CPU e memória, enquanto outros registram erros e descartes de pacotes na rede. Parte desses episódios termina com o retorno das métricas aos níveis normais. Esse histórico permite comparar os períodos com e sem alertas, identificar problemas que se repetem e acompanhar como cada episódio começa, se agrava e se recupera.

Às 20h30, há dois servidores críticos, dois em atenção e seis normais. Os quatro servidores afetados têm seis alertas ativos, porque um servidor pode ter alertas em mais de um componente. Há também um volume cujo uso cresce durante a semana, mesmo sem chegar ao limite de atenção. Esse caso será usado na previsão de capacidade.

O iFood é usado como referência de contexto. Os servidores e os dados são fictícios e não representam a estrutura real da empresa.

Preparação e leitura dos dados

Os dados foram gerados pela simulação do site e salvos em CSV. A análise usa o histórico da semana, o cadastro dos servidores, os limites de alerta, os volumes e as listas de processos e containers.

```
# Pasta com os arquivos exportados pelo site.
pasta_csv = "../../website/scripts/exportados/analise"

# Lê um dos arquivos dessa pasta.
ler_arquivo_csv = function(nome_arquivo) {
  read.csv(file.path(pasta_csv, nome_arquivo), stringsAsFactors = FALSE,
           fileEncoding = "UTF-8", na.strings = "")
}

historico_servidores = ler_arquivo_csv("historico-semana.csv")
cadastro_servidores = ler_arquivo_csv("servidores.csv")
limites_alerta = ler_arquivo_csv("limites.csv")
historico_volumes = ler_arquivo_csv("volumes-historico.csv")
processos_atuais = ler_arquivo_csv("processos-finais.csv")
containers_atuais = ler_arquivo_csv("containers-finais.csv")

# Converte o texto dos horários para data e hora de Brasília.
converter_horario = function(texto) {
  as.POSIXct(texto, format = "%Y-%m-%d %H:%M:%S", tz = "America/Sao_Paulo")
}

cadastro_servidores$mac = cadastro_servidores$ENDERECO_MAC
cadastro_servidores$docker = tolower(as.character(cadastro_servidores$docker_monitorado)) == "true"
historico_servidores$mac = historico_servidores$ENDERECO_MAC
```

```r
historico_servidores$instante = converter_horario(historico_servidores$TIMESTAMP)
historico_servidores = historico_servidores[order(historico_servidores$mac, historico_servidores$instant
rownames(historico_servidores) = NULL

inicio_semana = converter_horario("2026-10-12 20:30:00")
horario_final = converter_horario("2026-10-19 20:30:00")
horarios_semana = seq(inicio_semana, horario_final, by = "1 min")
periodos = c("30 minutos" = 30, "1 hora" = 60, "6 horas" = 360,
             "12 horas" = 720, "24 horas" = 1440, "7 dias" = 10080)
perfis_analisados = c("Todos", "Carrinho", "Financeiro", "Catálogo")
metricas_alerta = c("cpu_percent", "ram_percent", "disk_percent", "net_errors", "net_drops")

# Acrescenta nome, perfil e horário às tabelas de detalhe.
adicionar_cadastro = function(tabela) {
  tabela$mac = tabela$ENDERECO_MAC
  tabela$instante = converter_horario(tabela$TIMESTAMP)
  posicao_cadastro = match(tabela$mac, cadastro_servidores$mac)
  if (any(is.na(posicao_cadastro))) stop("Há um MAC que não pertence ao cadastro.")
  tabela$servidor = cadastro_servidores$servidor[posicao_cadastro]
  tabela$perfil = cadastro_servidores$perfil[posicao_cadastro]
  tabela
}
historico_volumes = adicionar_cadastro(historico_volumes)
processos_atuais = adicionar_cadastro(processos_atuais)
containers_atuais = adicionar_cadastro(containers_atuais)
volumes_finais = historico_volumes[historico_volumes$instante == horario_final, ]
```

Organização dos servidores e das observações

A base contém dez servidores, divididos entre Carrinho, Financeiro e Catálogo. Cada linha do histórico representa um servidor em um minuto. Os volumes ficam em uma tabela separada, pois um servidor pode ter mais de um volume. Processos e containers representam o instante final, às 20h30.

```r
knitr::kable(cadastro_servidores[c("servidor", "mac", "perfil", "cpu_logicas",
                        "ram_total_gb", "swap_total_gb", "docker")])
```

servidor	mac	perfil	cpu_logicas	ram_total_gb	swap_total_gb	docker
cart-api-07	AA:BB:CC:DD:EE:07	Carrinho	8	16	4	TRUE
cart-api-12	AA:BB:CC:DD:EE:12	Carrinho	8	16	4	TRUE
cart-api-02	AA:BB:CC:DD:EE:02	Carrinho	8	16	4	TRUE
cart-api-09	AA:BB:CC:DD:EE:09	Carrinho	8	16	4	TRUE
pay-db-02	AA:BB:CC:DD:EF:02	Financeiro	8	32	8	TRUE
pay-api-04	AA:BB:CC:DD:EF:04	Financeiro	8	16	4	TRUE
pay-api-06	AA:BB:CC:DD:EF:06	Financeiro	8	16	4	TRUE
catalog-api-03	AA:BB:CC:DD:F0:03	Catálogo	8	16	4	FALSE
catalog-api-05	AA:BB:CC:DD:F0:05	Catálogo	8	64	8	FALSE
catalog-api-08	AA:BB:CC:DD:F0:08	Catálogo	8	16	4	FALSE

```r
table(cadastro_servidores$perfil)
```

```
##
##   Carrinho   Catálogo Financeiro
##          4          3          3
```

```
knitr::kable(head(historico_servidores[c("servidor", "instante", metricas_alerta)]), digits = 2)
```

servidor	instante	cpu_percent	ram_percent	disk_percent	net_errors	net_drops
cart-api-02	2026-10-12 20:30:00	55.41	64.71	53.72	0.21	0.68
cart-api-02	2026-10-12 20:31:00	55.42	64.67	53.72	0.21	0.68
cart-api-02	2026-10-12 20:32:00	55.44	64.63	53.72	0.21	0.68
cart-api-02	2026-10-12 20:33:00	55.46	64.58	53.72	0.20	0.67
cart-api-02	2026-10-12 20:34:00	55.47	64.54	53.72	0.20	0.67
cart-api-02	2026-10-12 20:35:00	55.49	64.49	53.72	0.20	0.67

Qualidade dos dados

Antes dos cálculos, observamos a quantidade de registros, os valores ausentes e os horários repetidos. Um campo vazio não é tratado como zero. O histórico cobre sete dias com 10.081 registros por servidor, incluindo o primeiro e o último minuto.

```
resumo_dados = data.frame(
  informacao = c("Servidores", "Registros no histórico", "Horários repetidos", "Valores ausentes nas mé
  quantidade = c(nrow(cadastro_servidores), nrow(historico_servidores),
    sum(duplicated(historico_servidores[c("mac", "instante")])), sum(is.na(historico_servidores[metricas
knitr::kable(resumo_dados)
```

informacao	quantidade
Servidores	10
Registros no histórico	100810
Horários repetidos	0
Valores ausentes nas métricas de alerta	0

```
# Interrompe a análise se faltar uma métrica usada nos alertas.
if (any(is.na(historico_servidores[metricas_alerta]))) stop("Há valores ausentes nas métricas de alerta
if (any(is.na(historico_servidores$instante))) stop("Há horários que não puderam ser lidos.")
if (any(duplicated(cadastro_servidores$mac))) stop("Há MACs repetidos no cadastro.")
if (any(is.na(match(historico_servidores$mac, cadastro_servidores$mac)))) stop("Há MACs fora do cadastr

for (metrica_atual in metricas_alerta) {
  if (!is.numeric(historico_servidores[[metrica_atual]])) stop(paste("A coluna precisa ser numérica:", m
  if (any(!is.finite(historico_servidores[[metrica_atual]]) |
          historico_servidores[[metrica_atual]] < 0)) {
    stop(paste("Valor inválido em", metrica_atual))
  }
}
for (metrica_atual in c("cpu_percent", "ram_percent", "disk_percent", "swap_percent")) {
  if (any(is.na(historico_servidores[[metrica_atual]]) | historico_servidores[[metrica_atual]] < 0 | his
    stop(paste("Percentual inválido em", metrica_atual))
  }
```

```
}

for (mac_atual in cadastro_servidores$mac) {
  dados_servidor = historico_servidores[historico_servidores$mac == mac_atual, ]
  if (nrow(dados_servidor) != length(horarios_semana)) stop("Quantidade de registros diferente por servi
  if (any(duplicated(dados_servidor$instante))) stop("Há horários repetidos.")
  if (any(as.numeric(dados_servidor$instante) != as.numeric(horarios_semana))) stop("Há minutos faltando
}
knitr::kable(as.data.frame(table(historico_servidores$servidor)))
```

Var1	Freq
cart-api-02	10081
cart-api-07	10081
cart-api-09	10081
cart-api-12	10081
catalog-api-03	10081
catalog-api-05	10081
catalog-api-08	10081
pay-api-04	10081
pay-api-06	10081
pay-db-02	10081

Definição das métricas

O dicionário mostra as opções do catálogo de monitoramento. As métricas foram organizadas por recurso para explicar o que cada medida representa, sua unidade e o nível de detalhamento necessário pra análise.

Separamos medidas de uso dos recursos, sinais de espera e registros de problemas. Para entender um alerta, precisamos olhar o histórico e outras informações do servidor. Uma medida sozinha não explica a causa.

CPU

As métricas de CPU permitem observar a utilização do processamento e investigar diferenças entre CPUs lógicas e os tipos de atividade.

Métrica	Unidade	Finalidade
Uso total de CPU	%	Acompanhar a utilização do processamento do servidor
Uso por CPU lógica	% por CPU lógica	Identificar concentração de utilização que a média total pode esconder
CPU em modo usuário	%	Observar a parcela de tempo dedicada à execução em modo usuário
CPU em modo sistema	%	Observar a parcela de tempo dedicada à execução em modo kernel
CPU ociosa	%	Observar a parcela de tempo em que a CPU permanece ociosa

Métrica	Unidade	Finalidade
Espera por I/O	%	Apoiar a investigação de espera associada a operações de entrada e saída
Frequência da CPU	MHz	Acrescentar contexto sobre a frequência de operação
Interrupções	eventos/s	Acompanhar a atividade de interrupções do sistema
Trocas de contexto	eventos/s	Acompanhar a alternância de execução entre tarefas

A CPU total mostra o uso do servidor como um todo. O uso por CPU lógica mostra como esse trabalho se divide. Em processos e containers, 100% pode representar uma CPU lógica. Nesse caso, um consumidor que usa várias CPUs pode passar de 100%.

Carga do sistema

A carga média ajuda a entender quanto trabalho está sendo executado ou aguardando atendimento no servidor. No Linux, considera tanto tarefas executando ou aguardando CPU quanto tarefas em determinados estados de espera, como operações de disco. Por isso, carga alta não significa necessariamente CPU muito utilizada. O servidor pode ter várias tarefas esperando pelo disco, por exemplo, enquanto a CPU ainda tem capacidade disponível. Essa métrica deve ser interpretada junto ao uso de CPU e às métricas de I/O.

Métrica	Unidade	Finalidade
Carga média de 1 minuto	carga	Observar mudanças mais recentes na demanda
Carga média de 5 minutos	carga	Acompanhar a persistência da demanda
Carga média de 15 minutos	carga	Comparar o comportamento recente com uma janela mais longa

A carga não é um percentual. Para entendê-la, também olhamos a quantidade de CPUs lógicas e os sinais de CPU e disco.

Memória RAM

As métricas de RAM permitem observar ocupação, capacidade e margem disponível, além de ocorrências de encerramento por falta de memória.

Métrica	Unidade	Finalidade
Uso de RAM	%	Acompanhar a ocupação em relação à capacidade
Capacidade total de RAM	GB	Entender o percentual junto da quantidade em GB
Memória disponível	GB	Observar a estimativa de memória disponível para novas demandas

Métrica	Unidade	Finalidade
Memória em uso	GB	Mostrar a memória em uso conforme a definição da fonte
Memória livre	GB	Observar a memória não utilizada, distinguindo-a da disponível
Memória em cache	GB	Acrescentar contexto sobre o uso de memória para cache
Encerramentos por OOM	ocorrências no intervalo	Identificar novos encerramentos por falta de memória registrados pela fonte

Memória livre e memória disponível não são equivalentes. Não se deve assumir que memória em uso + memória disponível sempre corresponde à capacidade total.

Swap

A análise de swap complementa a observação da RAM, distinguindo ocupação de movimentação de dados.

Métrica	Unidade	Finalidade
Uso de swap	%	Acompanhar a proporção ocupada
Entrada de swap	bytes/s	Observar dados trazidos da swap para a RAM
Saída de swap	bytes/s	Observar dados enviados da RAM para a swap
Swap em uso	GB	Mostrar o espaço ocupado
Swap livre	GB	Mostrar o espaço restante

A ocupação da swap pode permanecer após um período de maior demanda e, sozinha, não indica falta de RAM no momento. Pra investigar pressão de memória, analisamos também a memória disponível e a movimentação de dados entre RAM e swap ao longo do tempo. Quando não houver swap configurada, essa condição será informada explicitamente.

Capacidade e atividade de armazenamento

Analisamos o espaço ocupado por volume, como `/dados`. Já as leituras e escritas são apresentadas por dispositivo. É preciso conhecer a relação entre eles antes de tratar os dois como a mesma coisa.

Métrica	Unidade	Detalhamento	Finalidade
Ocupação do volume	%	Volume	Acompanhar o consumo da capacidade
Espaço livre	GB	Volume	Mostrar a margem restante
Espaço ocupado	GB	Volume	Mostrar o armazenamento utilizado

Métrica	Unidade	Detalhamento	Finalidade
Uso de inodes	%	Sistema de arquivos aplicável	Observar o consumo da capacidade de manter arquivos
Tempo médio estimado de leitura e escrita	ms	Dispositivo e operação	Investigar o tempo médio das operações de I/O
Fila média de I/O	operações	Dispositivo	Acompanhar operações pendentes e em andamento conforme a fonte
Taxa de leitura	bytes/s	Dispositivo	Mostrar o volume de dados lidos por segundo
Taxa de escrita	bytes/s	Dispositivo	Mostrar o volume de dados escritos por segundo
Operações de leitura	IOPS	Dispositivo	Mostrar leituras por segundo
Operações de escrita	IOPS	Dispositivo	Mostrar escritas por segundo

A capacidade total do volume também será registrada como informação de referência.

Espaço disponível e desempenho de I/O respondem a perguntas diferentes: um volume pode ter espaço livre e apresentar operações lentas. O tempo médio estimado de I/O não corresponde à latência de uma requisição da aplicação.

Rede

As métricas de rede são identificadas por interface e, quando aplicável, pela direção de recebimento ou envio.

Métrica	Unidade	Finalidade
Dados recebidos e enviados	bytes/s	Acompanhar o tráfego em cada direção
Erros de recebimento e envio	eventos/s	Localizar ocorrências de erro na comunicação
Descartes na entrada e saída	pacotes/s	Acompanhar pacotes descartados
Estado da interface	ativa/inativa	Identificar o estado operacional informado pela fonte
Pacotes recebidos	pacotes/s	Acompanhar a quantidade de pacotes recebidos
Pacotes enviados	pacotes/s	Acompanhar a quantidade de pacotes enviados

Tráfego em bytes/s não representa percentual de utilização sem uma capacidade de referência. Erros e descartes exigem investigação e não comprovam, isoladamente, perda de pedidos ou falha de pagamento.

Processos e containers

Os processos e containers mostram quais programas estão usando os recursos do servidor.

Objeto	Medidas e informações	Finalidade
Processo	PID, nome, estado, CPU e RAM	Identificar consumidores de recursos dentro do servidor
Container	Identificação, estado, CPU, RAM e atividade de I/O	Investigar a distribuição do consumo entre containers

No cenário, o monitoramento Docker se aplica aos perfis Carrinho e Financeiro. No perfil Catálogo, a informação deve aparecer como "Não monitorado".

Um processo pode estar dentro de um container. Por isso, somar o consumo dos dois pode contar parte do uso duas vezes.

Indicadores calculados para a dashboard

Os indicadores são calculados com os valores das métricas, o cadastro e os limites de alerta.

Indicador	Definição
Servidores em atenção no período	Servidores cuja maior gravidade dentro do intervalo é atenção
Servidores críticos no período	Servidores cuja maior gravidade dentro do intervalo é crítica
Alertas do período	Episódios por servidor e componente ativos em algum trecho da janela
Alertas ativos	Condições por servidor e componente ainda abertas às 20h30
Evolução dos servidores em alerta	Contagem de servidores em atenção e críticos em cada instante
Servidores com alertas por recurso	Contagem de servidores distintos afetados em cada recurso, conforme a referência temporal definida
Previsão de capacidade	Prazo estimado até um limite de ocupação, considerando o histórico e a hipótese de crescimento adotada

Cards e barras da geral usam o período selecionado. As condições ativas e os valores atuais do detalhe usam o instante final. A previsão da geral usa o filtro; a previsão do detalhe usa a semana inteira.

Como escolhemos as métricas essenciais

Consideramos essencial uma métrica que faz falta para responder a uma pergunta importante da proposta do CoreTrace. Também observamos se ela pode ser calculada com outros campos e se faz sentido naquele ambiente.

Uma métrica pode aparecer na tela sem gerar alerta. Ter um campo no catálogo também não significa que ele precise ser usado em qualquer servidor.

Origem dos dados e método

Os arquivos contêm dados simulados de uma semana de monitoramento. A partir deles, calculamos a situação dos servidores, os incidentes, os resumos por período e as estatísticas de utilização. Os limites de alerta estão em `limites.csv` e representam a configuração usada na simulação.

Cada informação é analisada junto do servidor e do horário a que pertence. No disco, separamos os volumes e os dispositivos. Na rede, identificamos a interface. Assim, evitamos misturar valores de recursos diferentes.

Classificação por componente e por servidor

```
knitr::kable(limites_alerta)
```

perfil	metrica	atencao	critico	operador	origem
Carrinho	cpu_percent	70	90	>	limitesReferencia do gerador; conferir com MySQL
Carrinho	ram_percent	75	90	>	limitesReferencia do gerador; conferir com MySQL
Carrinho	disk_percent	80	95	>	limitesReferencia do gerador; conferir com MySQL
Carrinho	net_errors	5	15	>	limitesReferencia do gerador; conferir com MySQL
Carrinho	net_drops	10	30	>	limitesReferencia do gerador; conferir com MySQL
Financeiro	cpu_percent	75	92	>	limitesReferencia do gerador; conferir com MySQL
Financeiro	ram_percent	75	90	>	limitesReferencia do gerador; conferir com MySQL
Financeiro	disk_percent	80	95	>	limitesReferencia do gerador; conferir com MySQL
Financeiro	net_errors	5	15	>	limitesReferencia do gerador; conferir com MySQL
Financeiro	net_drops	10	30	>	limitesReferencia do gerador; conferir com MySQL
Catálogo	cpu_percent	75	90	>	limitesReferencia do gerador; conferir com MySQL
Catálogo	ram_percent	75	90	>	limitesReferencia do gerador; conferir com MySQL
Catálogo	disk_percent	80	95	>	limitesReferencia do gerador; conferir com MySQL
Catálogo	net_errors	5	15	>	limitesReferencia do gerador; conferir com MySQL
Catálogo	net_drops	10	30	>	limitesReferencia do gerador; conferir com MySQL

```
# Valores acima do limite crítico recebem a maior gravidade.
classificar = function(valor, atencao, critico) {
  ifelse(valor > critico, "Crítico", ifelse(valor > atencao, "Atenção", "Normal"))
}

# A posição indica a gravidade: Normal = 1, Atenção = 2 e Crítico = 3.
ordem_gravidade = c("Normal", "Atenção", "Crítico")
historico_servidores$gravidade = 1
for (metrica_atual in metricas_alerta) {
  coluna_situacao = paste0("estado_", metrica_atual)
  historico_servidores[[coluna_situacao]] = "Normal"
```

```
  for (perfil_atual in unique(historico_servidores$perfil)) {
    limite_metrica = limites_alerta[limites_alerta$perfil == perfil_atual & limites_alerta$metrica == me
    if (nrow(limite_metrica) != 1 || limite_metrica$operador != ">") stop("Confira os limites do perfil
    linhas_perfil = which(historico_servidores$perfil == perfil_atual)
    historico_servidores[linhas_perfil, coluna_situacao] = classificar(
      historico_servidores[linhas_perfil, metrica_atual],
      limite_metrica$atencao,
      limite_metrica$critico
    )
  }
  historico_servidores$gravidade = pmax(historico_servidores$gravidade, match(historico_servidores[[colu
}
historico_servidores$situacao = ordem_gravidade[historico_servidores$gravidade]
dados_atuais = historico_servidores[historico_servidores$instante == horario_final, ]
```

Neste cenário, cinco métricas têm limites de alerta. A situação do servidor é a mais grave entre seus componentes. Se ele está crítico, não entra também na contagem de servidores em atenção naquele minuto.

Estado dos servidores no instante final

```
knitr::kable(dados_atuais[c("servidor", "perfil", "cpu_percent", "ram_percent", "situacao")], digits = 1
```

	servidor	perfil	cpu_percent	ram_percent	situacao
10081	cart-api-02	Carrinho	57	64	Normal
20162	cart-api-07	Carrinho	94	81	Crítico
30243	cart-api-09	Carrinho	46	57	Normal
40324	cart-api-12	Carrinho	78	63	Atenção
50405	pay-db-02	Financeiro	65	79	Atenção
60486	pay-api-04	Financeiro	95	83	Crítico
70567	pay-api-06	Financeiro	58	62	Normal
80648	catalog-api-03	Catálogo	54	59	Normal
90729	catalog-api-05	Catálogo	51	66	Normal
100810	catalog-api-08	Catálogo	43	55	Normal

```
condicoes_atuais = data.frame()
for (metrica_atual in metricas_alerta) {
  condicoes_atuais = rbind(condicoes_atuais, data.frame(servidor = dados_atuais$servidor, mac = dados_a
    perfil = dados_atuais$perfil, metrica = metrica_atual, valor = dados_atuais[[metrica_atual]],
    situacao = dados_atuais[[paste0("estado_", metrica_atual)]]))
}
alertas_ativos = condicoes_atuais[condicoes_atuais$situacao != "Normal", ]
knitr::kable(alertas_ativos, digits = 2)
```

	servidor	mac	perfil	metrica	valor	situacao
2	cart-api-07	AA:BB:CC:DD:EE:07	Carrinho	cpu_percent	94	Crítico
4	cart-api-12	AA:BB:CC:DD:EE:12	Carrinho	cpu_percent	78	Atenção
6	pay-api-04	AA:BB:CC:DD:EF:04	Financeiro	cpu_percent	95	Crítico

	servidor	mac	perfil	metrica	valor	situacao
12	cart-api-07	AA:BB:CC:DD:EE:07	Carrinho	ram_percent	81	Atenção
15	pay-db-02	AA:BB:CC:DD:EF:02	Financeiro	ram_percent	79	Atenção
16	pay-api-04	AA:BB:CC:DD:EF:04	Financeiro	ram_percent	83	Atenção

```
indicadores_finais = data.frame(indicador = c("Servidores em atenção", "Servidores críticos",
  "Servidores normais", "Condições ativas", "Servidores afetados"),
  valor = c(sum(dados_atuais$situacao == "Atenção"), sum(dados_atuais$situacao == "Crítico"),
    sum(dados_atuais$situacao == "Normal"), nrow(alertas_ativos), length(unique(alertas_ativos$mac))))
knitr::kable(indicadores_finais)
```

indicador	valor
Servidores em atenção	2
Servidores críticos	2
Servidores normais	6
Condições ativas	6
Servidores afetados	4

Às 20h30, quatro servidores têm alertas: dois estão em atenção e dois estão críticos. Há seis condições ativas nesses quatro servidores. Os outros seis estão dentro dos limites das métricas analisadas. Isso, sozinho, não garante que toda a aplicação esteja funcionando bem.

Esta tabela final não é o card histórico da geral: ao escolher um período, os cards resumem o que ocorreu nesse intervalo, inclusive problemas que já terminaram.

Incidentes: início, escalada, máximo e recuperação

Consideramos um incidente o período em que uma métrica fica fora do normal sem se recuperar. Se ela passa de atenção para crítico, continua sendo o mesmo incidente. O fim é o primeiro registro em que ela volta ao normal.

```
incidentes = data.frame()
for (mac_atual in cadastro_servidores$mac) {
  dados_servidor = historico_servidores[historico_servidores$mac == mac_atual, ]
  for (metrica_atual in metricas_alerta) {
    situacoes_metrica = dados_servidor[[paste0("estado_", metrica_atual)]]
    alerta_ativo = situacoes_metrica != "Normal"
    # Início: agora está ativo, mas no minuto anterior não estava.
    alerta_minuto_anterior = c(FALSE, head(alerta_ativo, -1))
    alerta_minuto_seguinte = c(tail(alerta_ativo, -1), FALSE)
    indices_inicio = which(alerta_ativo & !alerta_minuto_anterior)
    indices_fim = which(alerta_ativo & !alerta_minuto_seguinte)
    for (indice_incidente in seq_along(indices_inicio)) {
      indice_inicio = indices_inicio[indice_incidente]
      indice_fim = indices_fim[indice_incidente]
      posicoes_criticas = which(situacoes_metrica[indice_inicio:indice_fim] == "Crítico")
      horario_escalada = as.POSIXct(NA, tz = "America/Sao_Paulo")
      if (length(posicoes_criticas) > 0) horario_escalada = dados_servidor$instante[indice_inicio + pos
      horario_fim_incidente = as.POSIXct(NA, tz = "America/Sao_Paulo")
      if (indice_fim < nrow(dados_servidor)) horario_fim_incidente = dados_servidor$instante[indice_fim
```

```
      incidentes = rbind(incidentes, data.frame(mac = mac_atual, servidor = dados_servidor$servidor[1],
        perfil = dados_servidor$perfil[1], metrica = metrica_atual, inicio = dados_servidor$instante[ini
        pico = max(dados_servidor[[metrica_atual]][indice_inicio:indice_fim]), fim = horario_fim_incide
        severidade = ifelse(length(posicoes_criticas) > 0, "Crítico", "Atenção")))
    }
  }
}
incidentes = incidentes[order(incidentes$inicio, incidentes$mac, incidentes$metrica), ]
rownames(incidentes) = NULL
knitr::kable(incidentes[c("servidor", "metrica", "inicio", "escalada", "pico", "fim", "severidade")], di
```

servidor	metrica	inicio	escalada	pico	fim	severidade
cart-api-02	ram_percent	2026-10-13 12:09:00	NA	80	2026-10-13 13:12:00	Atenção
catalog-api-03	cpu_percent	2026-10-13 12:14:00	NA	82	2026-10-13 13:16:00	Atenção
cart-api-09	cpu_percent	2026-10-14 18:42:00	NA	80	2026-10-14 20:07:00	Atenção
pay-api-06	cpu_percent	2026-10-14 19:03:00	NA	81	2026-10-14 20:11:00	Atenção
pay-db-02	ram_percent	2026-10-15 12:07:00	NA	84	2026-10-15 13:23:00	Atenção
catalog-api-03	cpu_percent	2026-10-15 12:13:00	NA	82	2026-10-15 13:17:00	Atenção
pay-api-06	cpu_percent	2026-10-16 18:58:00	2026-10-16 19:24:00	96	2026-10-16 20:38:00	Crítico
cart-api-02	cpu_percent	2026-10-16 19:07:00	NA	84	2026-10-16 20:33:00	Atenção
catalog-api-03	net_errors	2026-10-17 19:08:00	NA	9	2026-10-17 20:46:00	Atenção
cart-api-12	cpu_percent	2026-10-17 19:10:00	NA	80	2026-10-17 20:50:00	Atenção
catalog-api-03	net_drops	2026-10-17 19:17:00	NA	16	2026-10-17 20:43:00	Atenção
cart-api-07	cpu_percent	2026-10-18 18:58:00	NA	86	2026-10-18 21:12:00	Atenção
pay-db-02	ram_percent	2026-10-18 19:02:00	NA	82	2026-10-18 21:06:00	Atenção
catalog-api-03	cpu_percent	2026-10-19 12:09:00	NA	83	2026-10-19 13:13:00	Atenção
cart-api-02	ram_percent	2026-10-19 12:13:00	NA	82	2026-10-19 13:18:00	Atenção
cart-api-12	cpu_percent	2026-10-19 18:29:00	NA	78	NA	Atenção
pay-db-02	ram_percent	2026-10-19 18:44:00	NA	79	NA	Atenção
cart-api-07	cpu_percent	2026-10-19 18:56:00	2026-10-19 20:03:00	94	NA	Crítico
pay-api-04	cpu_percent	2026-10-19 19:07:00	2026-10-19 19:49:00	95	NA	Crítico
cart-api-07	ram_percent	2026-10-19 19:24:00	NA	81	NA	Atenção

servidor	metrica	inicio	escalada	pico	fim	severidade
pay-api-04	ram_percent	2026-10-19 19:29:00	NA	83	NA	Atenção

Um incidente pode aparecer no período mesmo que tenha começado antes, desde que ainda estivesse ativo em algum momento do intervalo. Se ele terminou exatamente no começo do filtro, não entra nessa contagem.

Incidentes no jantar de segunda-feira

A tabela apresenta o início, a escalada e o maior valor dos incidentes do jantar. O campo de encerramento fica vazio quando o incidente ainda estava ativo às 20h30.

```
incidentes_jantar = incidentes[incidentes$inicio >= converter_horario("2026-10-19 14:30:00"), ]
knitr::kable(incidentes_jantar[c("servidor", "metrica", "inicio", "escalada", "pico", "fim", "severidade
```

	servidor	metrica	inicio	escalada	pico	fim	severidade
16	cart-api-12	cpu_percent	2026-10-19 18:29:00	NA	78	NA	Atenção
17	pay-db-02	ram_percent	2026-10-19 18:44:00	NA	79	NA	Atenção
18	cart-api-07	cpu_percent	2026-10-19 18:56:00	2026-10-19 20:03:00	94	NA	Crítico
19	pay-api-04	cpu_percent	2026-10-19 19:07:00	2026-10-19 19:49:00	95	NA	Crítico
20	cart-api-07	ram_percent	2026-10-19 19:24:00	NA	81	NA	Atenção
21	pay-api-04	ram_percent	2026-10-19 19:29:00	NA	83	NA	Atenção

Filtros, cards e barras da visão geral

Os filtros mostram partes do histórico que terminam às 20h30. Por exemplo, o filtro de uma hora usa os registros de 19h30 a 20h30. Todos usam a mesma semana de dados.

Nos cards, cada servidor é contado uma vez, pela situação mais grave que teve no período. O card de alertas conta os incidentes por métrica que passaram pelo intervalo, incluindo os que começaram antes. Para essa contagem, vale a maior gravidade dentro do período escolhido.

```
# Tamanho das margens e dos rótulos do gráfico.
par(mar = c(5.5, 4.5, 3.5, 1.5), mgp = c(2.5, 0.8, 0), cex = 0.9)
# Seleciona os registros do período e do perfil.
selecionar_periodo = function(periodo_atual, perfil = "Todos") {
  inicio_periodo = horario_final - periodos[periodo_atual] * 60
  dados_periodo = historico_servidores[
    historico_servidores$instante >= inicio_periodo &
    historico_servidores$instante <= horario_final, ]
  if (perfil != "Todos") dados_periodo = dados_periodo[dados_periodo$perfil == perfil, ]
  dados_periodo
}

# Calcula máximos por servidor e conta episódios dentro da janela.
# Ao começar uma janela no meio de um episódio, ele é contado uma vez.
resumir_periodo = function(periodo_atual, perfil = "Todos") {
  dados_periodo = selecionar_periodo(periodo_atual, perfil)
  servidores_atencao = 0
```

```
  servidores_criticos = 0
  episodios_atencao = 0
  episodios_criticos = 0
  for (mac_atual in unique(dados_periodo$mac)) {
    dados_servidor_periodo = dados_periodo[dados_periodo$mac == mac_atual, ]
    maior_gravidade_servidor = max(dados_servidor_periodo$gravidade)
    if (maior_gravidade_servidor == 2) servidores_atencao = servidores_atencao + 1
    if (maior_gravidade_servidor == 3) servidores_criticos = servidores_criticos + 1
    for (metrica_atual in metricas_alerta) {
      gravidades_metrica = match(dados_servidor_periodo[[paste0("estado_", metrica_atual)]], ordem_gravi
      alerta_ativo = gravidades_metrica > 1
      alerta_minuto_anterior = c(FALSE, head(alerta_ativo, -1))
      alerta_minuto_seguinte = c(tail(alerta_ativo, -1), FALSE)
      indices_inicio = which(alerta_ativo & !alerta_minuto_anterior)
      indices_fim = which(alerta_ativo & !alerta_minuto_seguinte)
      for (indice_episodio in seq_along(indices_inicio)) {
        maior_gravidade = max(gravidades_metrica[indices_inicio[indice_episodio]:indices_fim[indice_epis
        if (maior_gravidade == 3) episodios_criticos = episodios_criticos + 1
        if (maior_gravidade == 2) episodios_atencao = episodios_atencao + 1
      }
    }
  }
  data.frame(periodo = periodo_atual, perfil = perfil,
    servidores_atencao = servidores_atencao, servidores_criticos = servidores_criticos,
    episodios_atencao = episodios_atencao, episodios_criticos = episodios_criticos,
    alertas_no_periodo = episodios_atencao + episodios_criticos)
}

cards_periodo = data.frame()
for (perfil_atual in perfis_analisados) {
  for (periodo_atual in names(periodos)) {
    cards_periodo = rbind(cards_periodo, resumir_periodo(periodo_atual, perfil_atual))
  }
}
knitr::kable(cards_periodo)
```

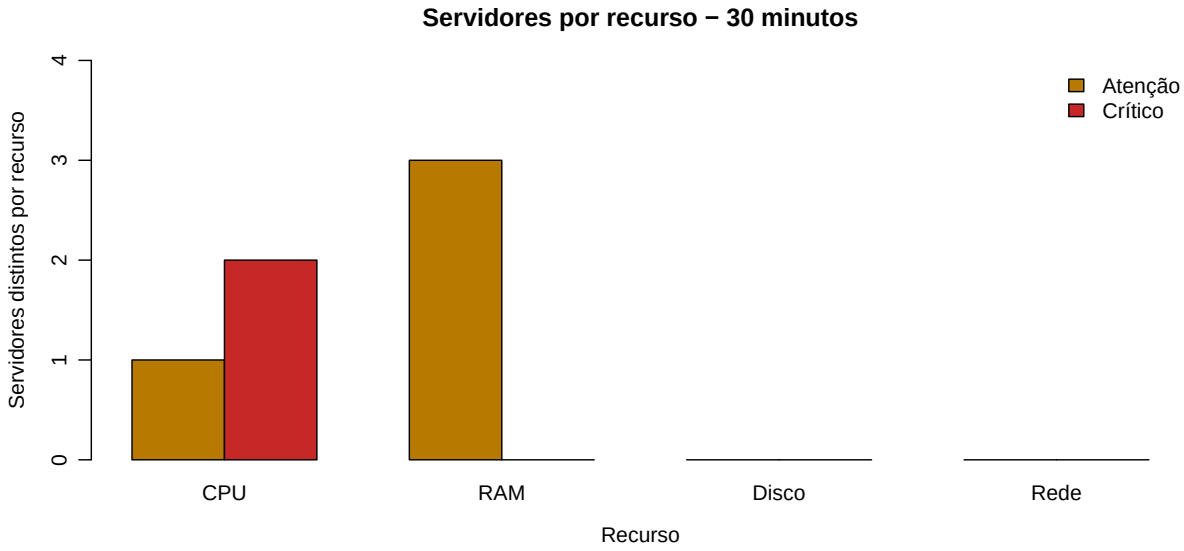
periodo	perfil	servidores_atencao	servidores_criticos	episodios_atencao	episodios_criticos	alertas_no_periodo
30 minutos	Todos	2	2	4	2	6
1 hora	Todos	2	2	4	2	6
6 horas	Todos	2	2	4	2	6
12 horas	Todos	4	2	6	2	8
24 horas	Todos	4	2	8	2	10
7 dias	Todos	5	3	18	3	21
30 minutos	Carrinho	1	1	2	1	3
1 hora	Carrinho	1	1	2	1	3
6 horas	Carrinho	1	1	2	1	3
12 horas	Carrinho	2	1	3	1	4
24 horas	Carrinho	2	1	4	1	5
7 dias	Carrinho	3	1	8	1	9

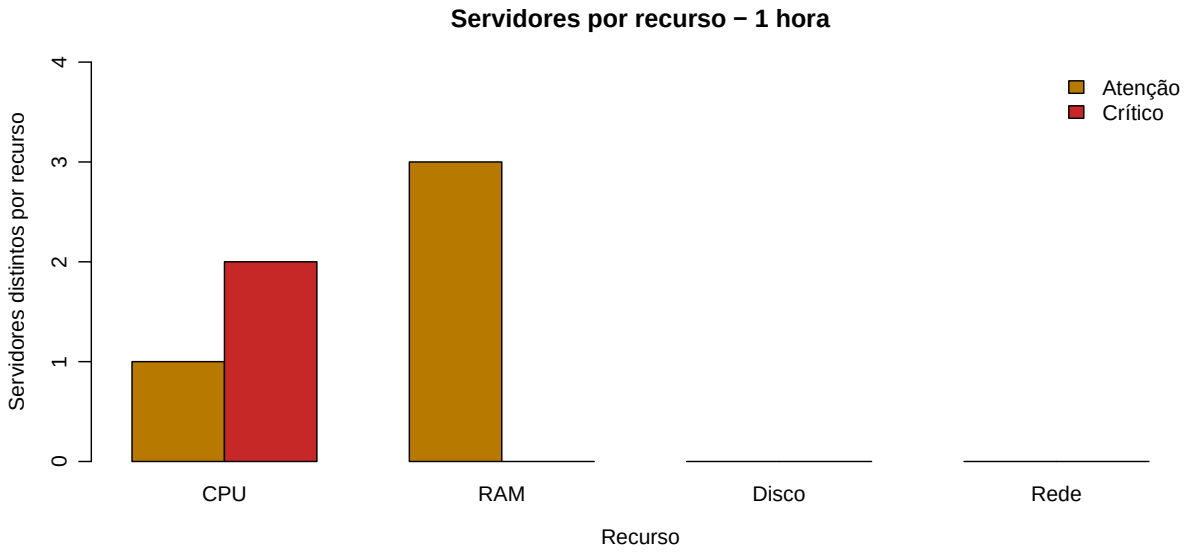
periodo	perfil	servidores_atencao	servidores_criticos	episodios_atencao	episodios_criticos	alertas_no_periodo
30 minutos	Financeiro	1	1	2	1	3
1 hora	Financeiro	1	1	2	1	3
6 horas	Financeiro	1	1	2	1	3
12 horas	Financeiro	1	1	2	1	3
24 horas	Financeiro	1	1	3	1	4
7 dias	Financeiro	1	2	5	2	7
30 minutos	Catálogo	0	0	0	0	0
1 hora	Catálogo	0	0	0	0	0
6 horas	Catálogo	0	0	0	0	0
12 horas	Catálogo	1	0	1	0	1
24 horas	Catálogo	1	0	1	0	1
7 dias	Catálogo	1	0	5	0	5

```
recursos_analisados = c("CPU", "RAM", "Disco", "Rede")
barras_periodo = data.frame()
for (perfil_atual in perfis_analisados) {
  for (periodo_atual in names(periodos)) {
    dados_periodo = selecionar_periodo(periodo_atual, perfil_atual)
    for (recurso in recursos_analisados) {
      metricas_recurso = switch(recurso, CPU = "cpu_percent", RAM = "ram_percent",
        Disco = "disk_percent", Rede = c("net_errors", "net_drops"))
      quantidade_atencao = 0
      quantidade_criticos = 0
      for (mac_atual in unique(dados_periodo$mac)) {
        dados_servidor_periodo = dados_periodo[dados_periodo$mac == mac_atual, ]
        maior_gravidade = 1
        for (metrica_atual in metricas_recurso) {
          coluna_situacao = paste0("estado_", metrica_atual)
          gravidades_recurso = match(dados_servidor_periodo[[coluna_situacao]], ordem_gravidade)
          maior_gravidade = max(maior_gravidade, gravidades_recurso)
        }
        if (maior_gravidade == 2) quantidade_atencao = quantidade_atencao + 1
        if (maior_gravidade == 3) quantidade_criticos = quantidade_criticos + 1
      }
      barras_periodo = rbind(barras_periodo, data.frame(periodo = periodo_atual,
        perfil = perfil_atual, recurso = recurso, atencao = quantidade_atencao, critico = quantidade_cri
    }
  }
}

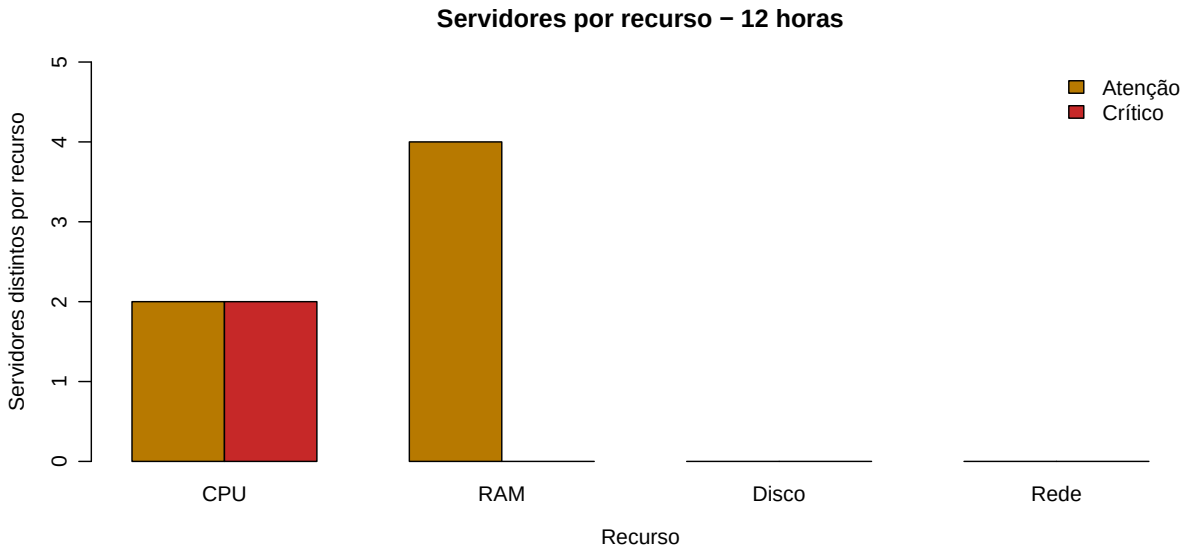
# Exibe os seis gráficos gerais. A tabela também contém os três perfis.
for (periodo_atual in names(periodos)) {
  barras_periodo_atual = barras_periodo[barras_periodo$periodo == periodo_atual & barras_periodo$perfil
  contagem_recursos = rbind(Atenção = barras_periodo_atual$atencao, Crítico = barras_periodo_atual$criti
  colnames(contagem_recursos) = barras_periodo_atual$recurso
  barplot(contagem_recursos, beside = TRUE, col = c("#B77900", "#C62828"),
    ylim = c(0, max(4, contagem_recursos + 1)), main = paste("Servidores por recurso -", periodo_atual)
    ylab = "Servidores distintos por recurso", xlab = "Recurso",
    legend.text = rownames(contagem_recursos), args.legend = list(x = "topright", bty = "n"))
```

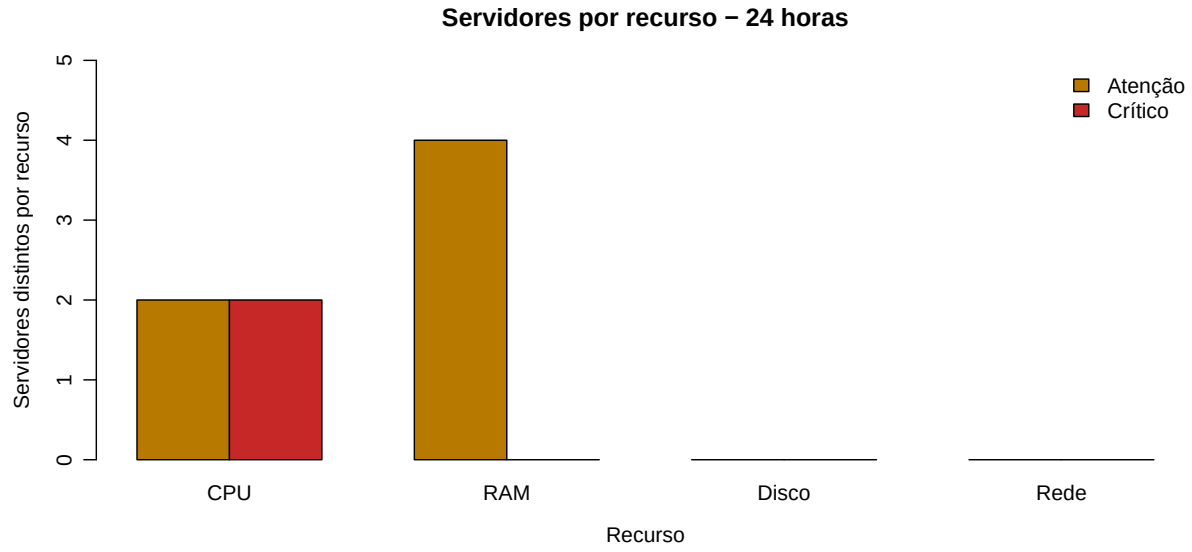
```
}
```

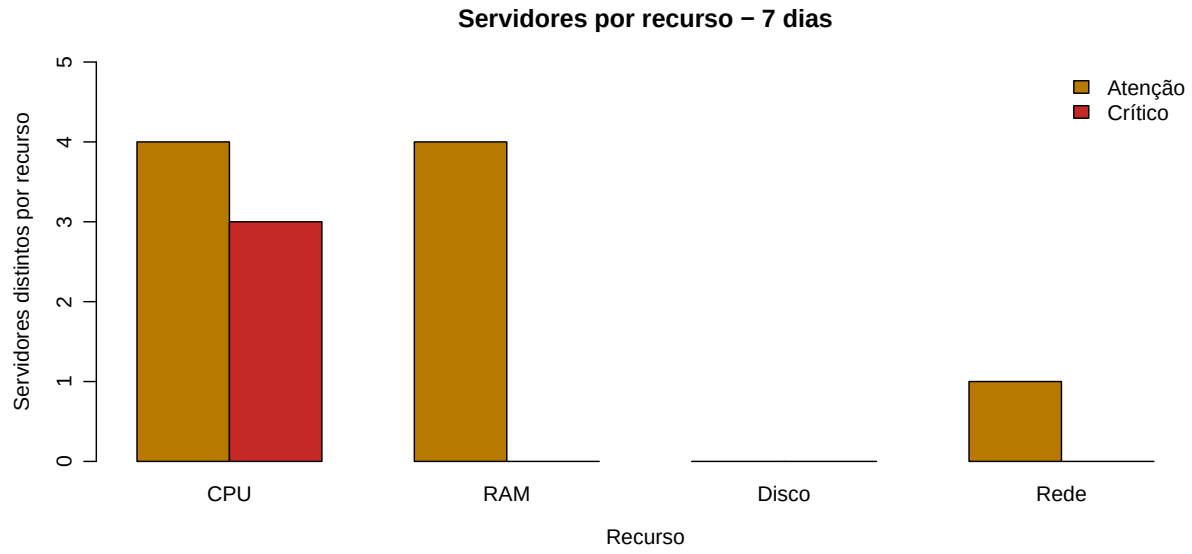

Servidores por recurso − 30 minutos
Servidores distintos por recurso
0
1
2
3
4
CPU
RAM
Disco
Rede
Recurso
Atenção
Crítico


Servidores por recurso − 1 hora
Servidores distintos por recurso
0
1
2
3
4
CPU
RAM
Disco
Rede
Recurso
Atenção
Crítico


Servidores por recurso − 6 horas
Servidores distintos por recurso
0
1
2
3
4
Atenção
Crítico
CPU
RAM
Disco
Rede
Recurso


Servidores por recurso − 12 horas
Servidores distintos por recurso
0
1
2
3
4
5
Atenção
Crítico
CPU
RAM
Disco
Rede
Recurso






```
knitr::kable(barras_periodo[barras_periodo$perfil == "Todos", ])
```

periodo	perfil	recurso	atencao	critico
30 minutos	Todos	CPU	1	2
30 minutos	Todos	RAM	3	0
30 minutos	Todos	Disco	0	0
30 minutos	Todos	Rede	0	0
1 hora	Todos	CPU	1	2
1 hora	Todos	RAM	3	0
1 hora	Todos	Disco	0	0
1 hora	Todos	Rede	0	0
6 horas	Todos	CPU	1	2

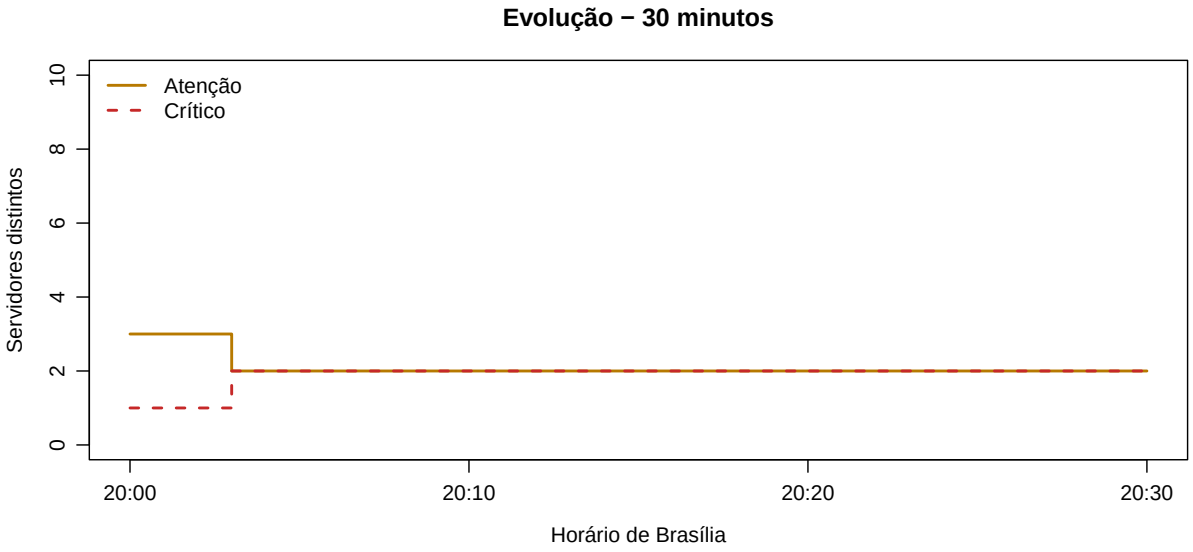
periodo	perfil	recurso	atencao	critico
6 horas	Todos	RAM	3	0
6 horas	Todos	Disco	0	0
6 horas	Todos	Rede	0	0
12 horas	Todos	CPU	2	2
12 horas	Todos	RAM	4	0
12 horas	Todos	Disco	0	0
12 horas	Todos	Rede	0	0
24 horas	Todos	CPU	2	2
24 horas	Todos	RAM	4	0
24 horas	Todos	Disco	0	0
24 horas	Todos	Rede	0	0
7 dias	Todos	CPU	4	3
7 dias	Todos	RAM	4	0
7 dias	Todos	Disco	0	0
7 dias	Todos	Rede	1	0

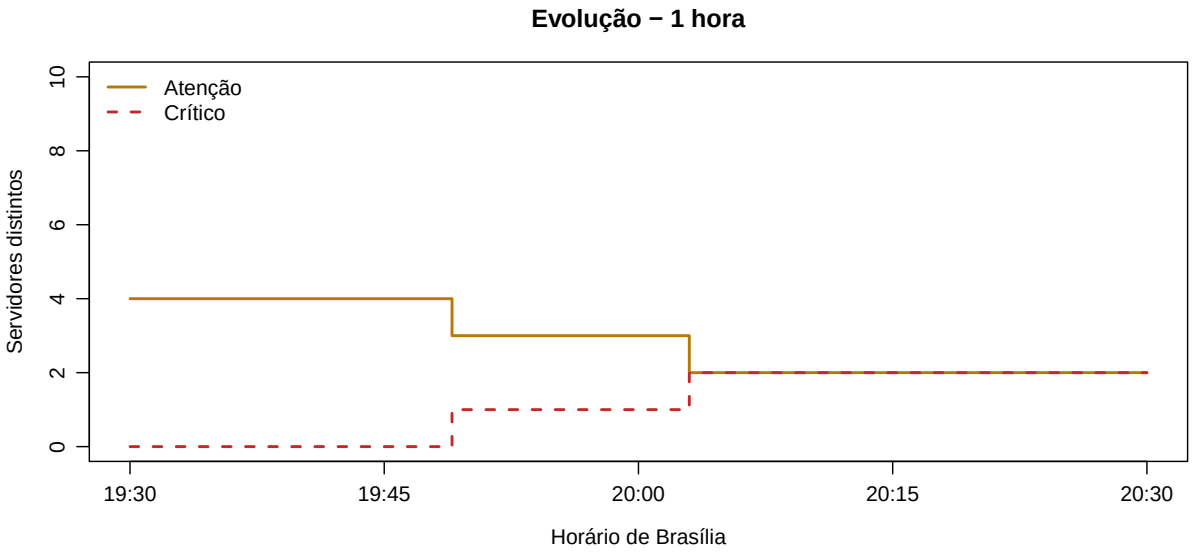
As barras ficam lado a lado, como na dash. Para cada recurso, o servidor aparece na maior gravidade do período. Erros e descartes da mesma interface entram juntos em Rede sem contar o servidor duas vezes. Um servidor pode aparecer em CPU e RAM; somar recursos não dá servidores únicos.

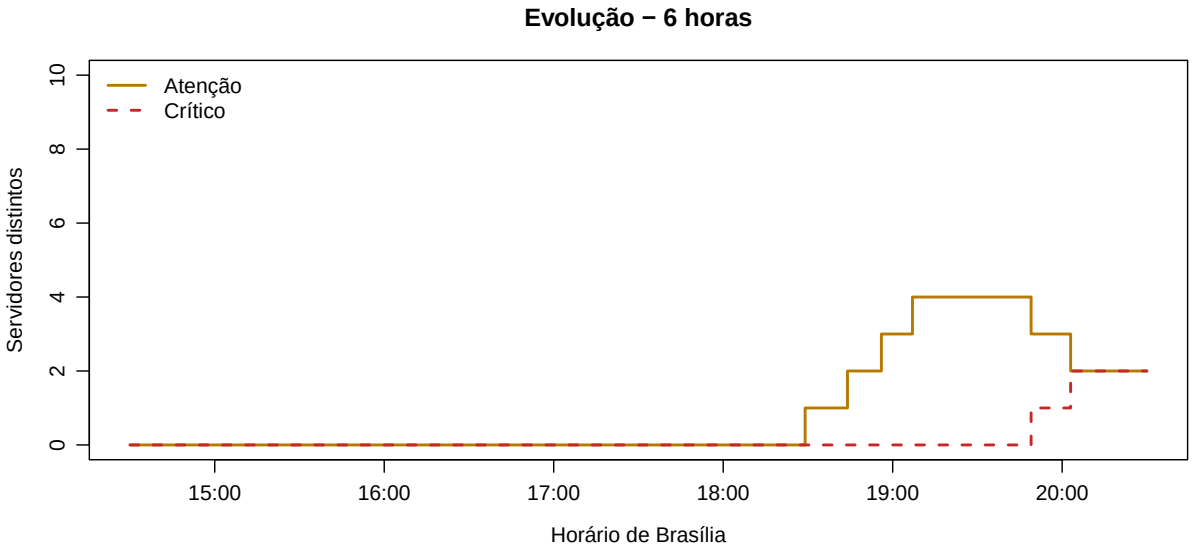
Condições ativas às 20h30 e episódios do período são contagens diferentes. Um incidente pode ter terminado antes do instante final e ainda aparecer no resumo da semana.

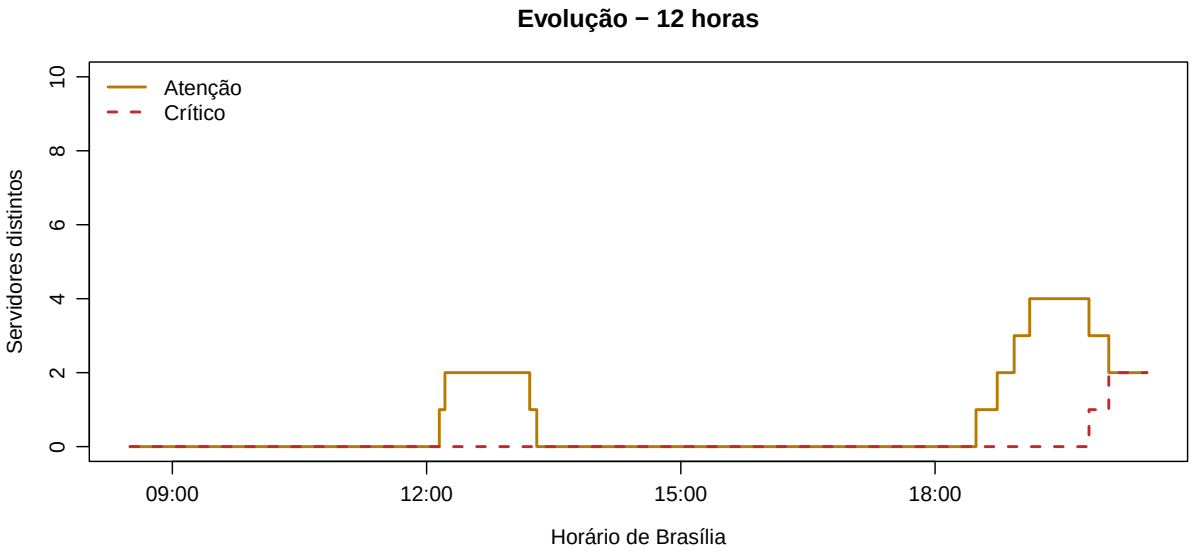
Evolução dos servidores em alerta

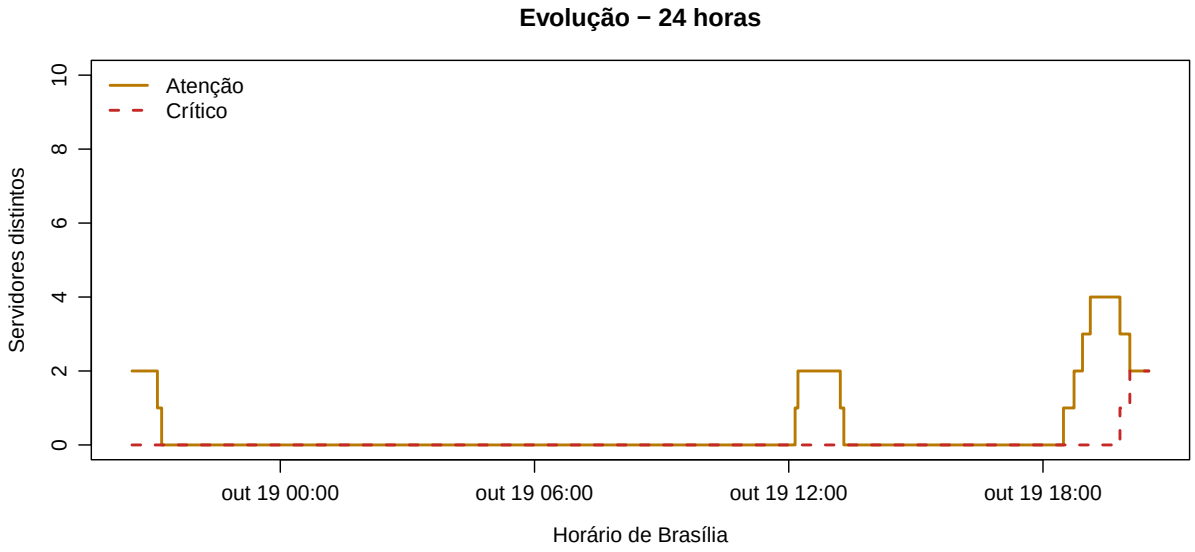
```
# Tamanho das margens e dos rótulos do gráfico.
par(mar = c(5.5, 4.5, 3.5, 1.5), mgp = c(2.5, 0.8, 0), cex = 0.9)
evolucao = data.frame(instante = horarios_semana, atencao = 0, critico = 0)
for (mac_atual in cadastro_servidores$mac) {
  situacoes_servidor = historico_servidores$situacao[historico_servidores$mac == mac_atual]
  evolucao$atencao = evolucao$atencao + as.integer(situacoes_servidor == "Atenção")
  evolucao$critico = evolucao$critico + as.integer(situacoes_servidor == "Crítico")
}
for (periodo_atual in names(periodos)) {
  evolucao_periodo = evolucao[evolucao$instante >= horario_final - periodos[periodo_atual] * 60, ]
  plot(evolucao_periodo$instante, evolucao_periodo$atencao, type = "s", col = "#B77900", lwd = 2,
    ylim = c(0, 10), xlab = "Horário de Brasília", ylab = "Servidores distintos",
    main = paste("Evolução -", periodo_atual))
  lines(evolucao_periodo$instante, evolucao_periodo$critico, type = "s", col = "#C62828", lwd = 2, lty =
  legend("topleft", c("Atenção", "Crítico"), col = c("#B77900", "#C62828"),
    lty = c(1, 2), lwd = 2, bty = "n")
}
```

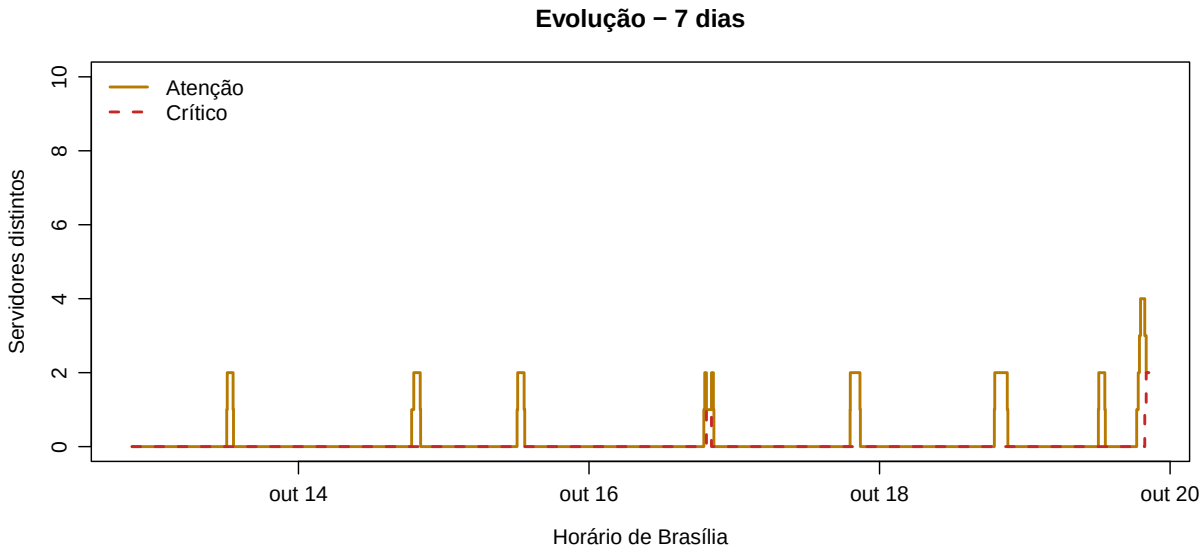

Evolução − 30 minutos
Atenção
Crítico
Servidores distintos
0
2
4
6
8
10
20:00
20:10
20:20
20:30
Horário de Brasília


Evolução − 1 hora
Atenção
Crítico
Servidores distintos
0
2
4
6
8
10
19:30
19:45
20:00
20:15
20:30
Horário de Brasília


Evolução − 6 horas
Atenção
Crítico
Servidores distintos
0
2
4
6
8
10
15:00
16:00
17:00
18:00
19:00
20:00
Horário de Brasília


Evolução − 12 horas
Atenção
Crítico
Servidores distintos
0
2
4
6
8
10
09:00
12:00
15:00
18:00
Horário de Brasília






```r
# Servidores com alerta no trecho de domingo incluído nas últimas 24 horas.
domingo_24h = historico_servidores[historico_servidores$instante >= horario_final - 1440 * 60 &
  historico_servidores$instante < converter_horario("2026-10-19 00:00:00") & historico_servidores$situa
knitr::kable(data.frame(servidor = unique(domingo_24h$servidor)))
```

servidor
cart-api-07
pay-db-02

```r
# O filtro inclui os incidentes ainda ativos, mesmo iniciados antes dele.
incidentes_30m = incidentes[incidentes$inicio <= horario_final &
```

```
  (is.na(incidentes$fim) | incidentes$fim > horario_final - 30 * 60), ]
knitr::kable(incidentes_30m[c("servidor", "metrica", "inicio", "escalada", "pico", "fim", "severidade")]
```

	servidor	metrica	inicio	escalada	pico	fim	severidade
16	cart-api-12	cpu_percent	2026-10-19 18:29:00	NA	78	NA	Atenção
17	pay-db-02	ram_percent	2026-10-19 18:44:00	NA	79	NA	Atenção
18	cart-api-07	cpu_percent	2026-10-19 18:56:00	2026-10-19 20:03:00	94	NA	Crítico
19	pay-api-04	cpu_percent	2026-10-19 19:07:00	2026-10-19 19:49:00	95	NA	Crítico
20	cart-api-07	ram_percent	2026-10-19 19:24:00	NA	81	NA	Atenção
21	pay-api-04	ram_percent	2026-10-19 19:29:00	NA	83	NA	Atenção

A linha em degraus muda no minuto registrado. Quando um servidor passa de atenção para crítico, ele sai de uma contagem e entra na outra. A contagem em atenção pode cair enquanto a gravidade aumenta.

Período	Pergunta apoiada
30 minutos	Como ocorreu a escalada crítica final?
1 hora	Quais mudanças antecederam o estado atual?
6 horas	Como o jantar se diferencia da tarde?
12 horas	Como almoço e jantar se comparam?
24 horas	Qual parte do episódio de domingo ainda aparece?
7 dias	Quais episódios se repetem e quais são diferentes?

Os filtros mostram partes da mesma história. Como são só sete dias simulados, não podemos afirmar que esse padrão se repete numa operação real. Um registro por minuto também pode deixar de mostrar acontecimentos mais rápidos.

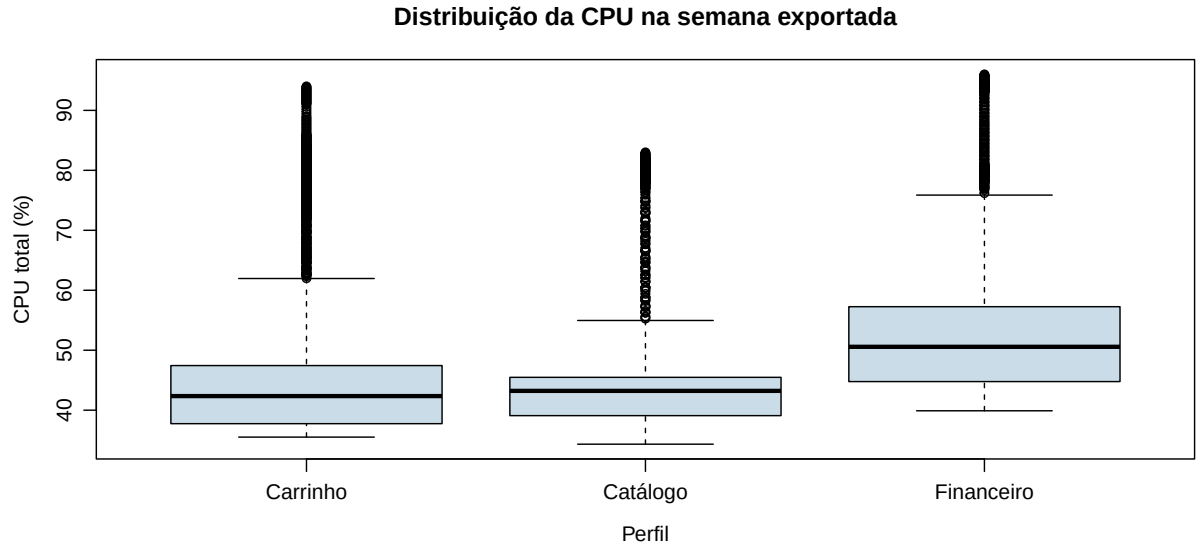
Distribuição e persistência

```
# Tamanho das margens e dos rótulos do gráfico.
par(mar = c(5.5, 4.5, 3.5, 1.5), mgp = c(2.5, 0.8, 0), cex = 0.9)
estatisticas_servidores = data.frame()
for (nome_servidor in cadastro_servidores$servidor) {
  dados_servidor = historico_servidores[historico_servidores$servidor == nome_servidor, ]
  estatisticas_servidores = rbind(estatisticas_servidores, data.frame(servidor = nome_servidor,
    cpu_media = mean(dados_servidor$cpu_percent), cpu_mediana = median(dados_servidor$cpu_percent),
    cpu_p95 = as.numeric(quantile(dados_servidor$cpu_percent, 0.95)), cpu_max = max(dados_servidor$cpu_
    ram_max = max(dados_servidor$ram_percent), amostras_atencao = sum(dados_servidor$situacao == "Atenç
    amostras_criticas = sum(dados_servidor$situacao == "Crítico")))
}
knitr::kable(estatisticas_servidores, digits = 2)
```

servidor	cpu_media	cpu_mediana	cpu_p95	cpu_max	ram_max	amostras_atencao	amostras_criticas
cart-api-07	44.88	42.99	50.87	94.00	81.00	201	28
cart-api-12	39.87	37.99	45.77	80.00	63.39	222	0
cart-api-02	49.99	48.72	55.86	84.00	82.00	214	0
cart-api-09	38.98	37.60	44.86	80.00	58.67	85	0

servidor	cpu_media	cpu_mediana	cpu_p95	cpu_max	ram_max	amostras_atencao	amostras_criticas
pay-db-02	59.01	57.94	65.07	67.05	84.00	307	0
pay-api-04	43.47	41.99	49.38	95.00	83.00	42	42
pay-api-06	52.08	50.52	57.98	96.00	62.71	118	50
catalog-api-03	47.17	45.44	53.54	83.00	60.56	288	0
catalog-api-05	44.15	43.09	50.20	52.14	67.46	0	0
catalog-api-08	37.48	36.42	43.55	45.47	55.61	0	0

```
boxplot(cpu_percent ~ perfil, data = historico_servidores, col = "#C9DCE8",
  main = "Distribuição da CPU na semana exportada", xlab = "Perfil", ylab = "CPU total (%)")
```



A média e a mediana ajudam a resumir o uso de CPU, mas podem esconder alertas curtos. O percentil 95 mostra o valor abaixo do qual ficam cerca de 95% dos registros. O máximo mostra o maior uso observado. Os pontos altos do boxplot fazem parte dos episódios simulados e não foram retirados. Cada minuto é um registro da mesma semana, e não um experimento separado.

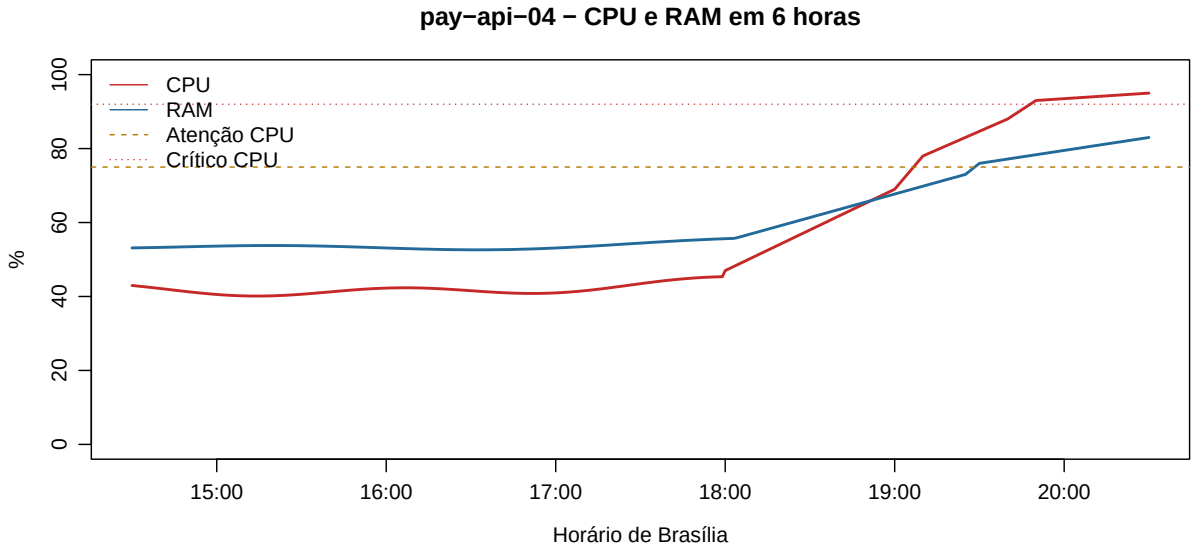
Investigação técnica dos recursos

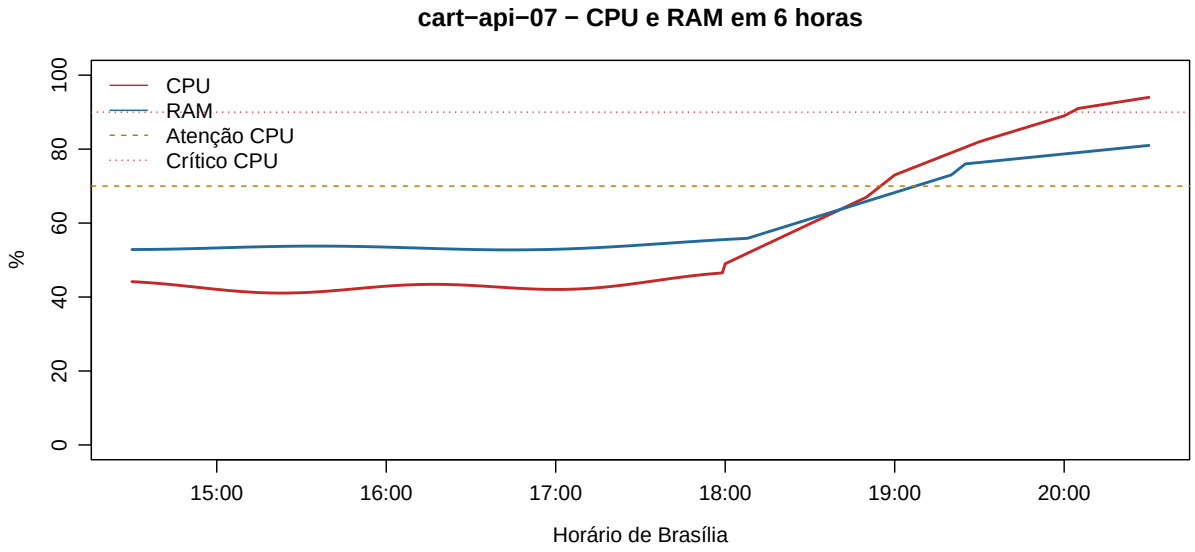
Referência temporal da tela de detalhe

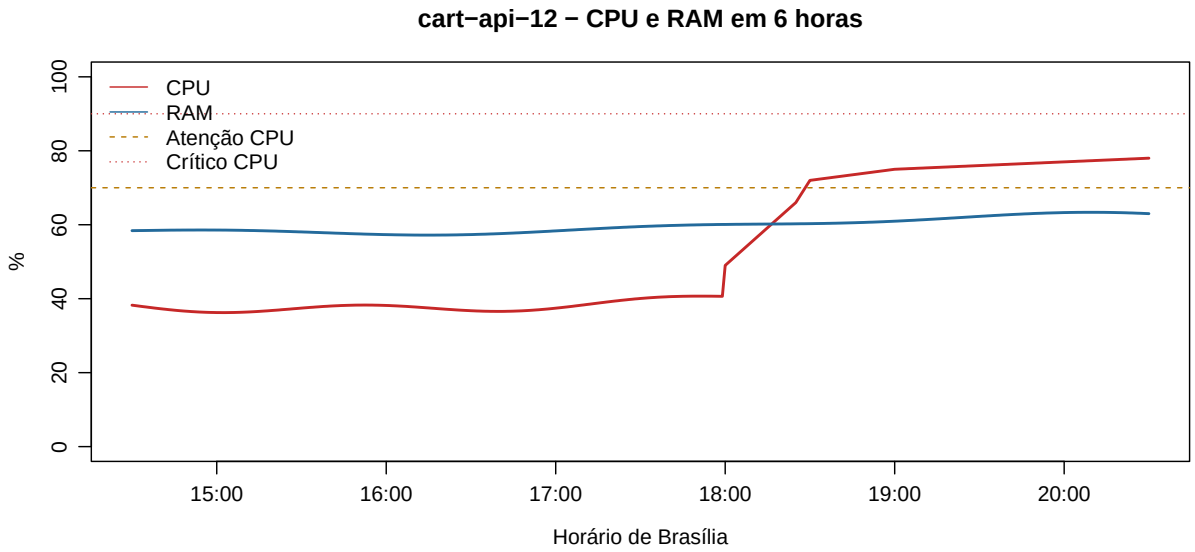
No detalhe, gráficos e incidentes usam o período escolhido. CPU/RAM em destaque, memória atual, processos, containers e volumes atuais continuam representando 20h30. Os medidores de CPU lógica usam médias do período. A previsão do detalhe usa a semana toda. Assim, o período descreve a evolução dos sinais, enquanto os valores atuais mostram a situação às 20h30.

CPU total, CPU lógica e memória

```r
# Tamanho das margens e dos rótulos do gráfico.
par(mar = c(5.5, 4.5, 3.5, 1.5), mgp = c(2.5, 0.8, 0), cex = 0.9)
for (nome_servidor in c("pay-api-04", "cart-api-07", "cart-api-12")) {
  dados_servidor = historico_servidores[historico_servidores$servidor == nome_servidor & historico_servi
  limite_cpu = limites_alerta[limites_alerta$perfil == dados_servidor$perfil[1] & limites_alerta$metrica
  plot(dados_servidor$instante, dados_servidor$cpu_percent, type = "l", lwd = 2, col = "#C62828", ylim =
    main = paste(nome_servidor, "- CPU e RAM em 6 horas"), xlab = "Horário de Brasília", ylab = "%")
  lines(dados_servidor$instante, dados_servidor$ram_percent, col = "#236A9B", lwd = 2)
  abline(h = c(limite_cpu$atencao, limite_cpu$critico), lty = c(2, 3), col = c("#B77900", "#C62828"))
  legend("topleft", c("CPU", "RAM", "Atenção CPU", "Crítico CPU"),
    col = c("#C62828", "#236A9B", "#B77900", "#C62828"), lty = c(1, 1, 2, 3), bty = "n")
}
```

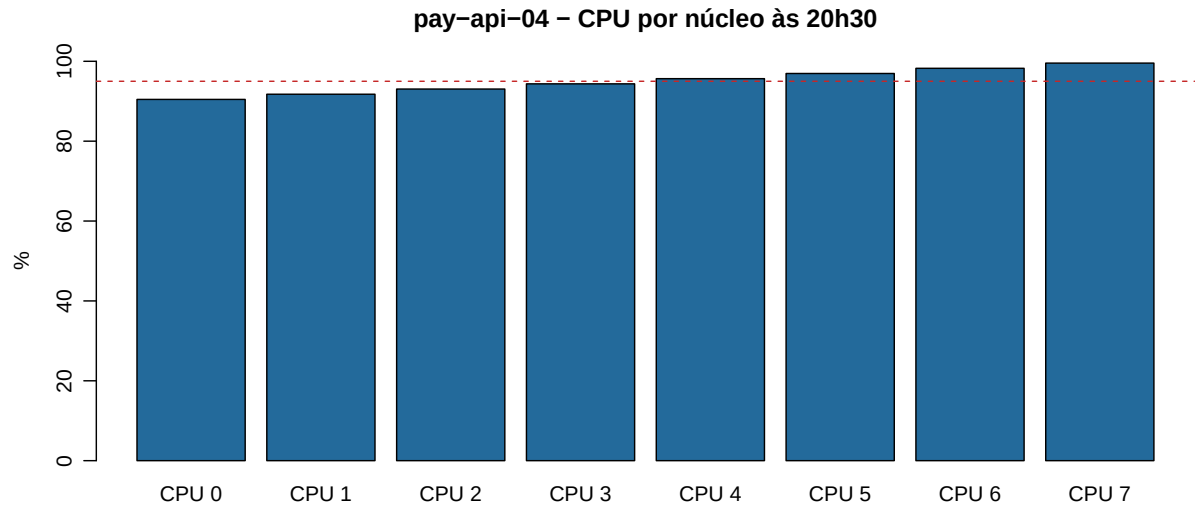







```r
# Valores por núcleo e médias de cada período.
medias_nucleos = data.frame()
for (indice_servidor in seq_len(nrow(cadastro_servidores))) {
  servidor_atual = cadastro_servidores[indice_servidor, ]
  colunas_nucleos = paste0("cpu_percent_percpu_", 0:(servidor_atual$cpu_logicas - 1))
  dados_servidor = historico_servidores[historico_servidores$mac == servidor_atual$mac, ]
  for (periodo_atual in names(periodos)) {
    dados_periodo = dados_servidor[dados_servidor$instante >= horario_final - periodos[periodo_atual] *
    medias_por_nucleo = numeric(length(colunas_nucleos))
    for (indice_nucleo in seq_along(colunas_nucleos)) {
      medias_por_nucleo[indice_nucleo] = mean(dados_periodo[[colunas_nucleos[indice_nucleo]]])
    }
    medias_nucleos = rbind(medias_nucleos, data.frame(servidor = servidor_atual$servidor,
      periodo = periodo_atual, cpu = 0:(servidor_atual$cpu_logicas - 1), media_percent = round(medias_p
```

```
  }
}
coleta_pay_api04 = dados_atuais[dados_atuais$servidor == "pay-api-04", ]
colunas_nucleos = paste0("cpu_percent_percpu_", 0:(coleta_pay_api04$cpu_count_logical - 1))
valores_nucleos = as.numeric(coleta_pay_api04[1, colunas_nucleos])
barplot(valores_nucleos, names.arg = paste0("CPU ", 0:(length(valores_nucleos) - 1)), ylim = c(0, 100),
  col = "#236A9B", main = "pay-api-04 - CPU por núcleo às 20h30", ylab = "%")
abline(h = coleta_pay_api04$cpu_percent, col = "#C62828", lty = 2)
```




```
knitr::kable(medias_nucleos[medias_nucleos$servidor == "pay-api-04", ], digits = 2)
```

	servidor	periodo	cpu	media_percent
241	pay-api-04	30 minutos	0	89.02
242	pay-api-04	30 minutos	1	90.52
243	pay-api-04	30 minutos	2	92.01
244	pay-api-04	30 minutos	3	93.50
245	pay-api-04	30 minutos	4	95.00
246	pay-api-04	30 minutos	5	96.49
247	pay-api-04	30 minutos	6	97.98
248	pay-api-04	30 minutos	7	99.48
249	pay-api-04	1 hora	0	86.14
250	pay-api-04	1 hora	1	87.85
251	pay-api-04	1 hora	2	89.55
252	pay-api-04	1 hora	3	91.25
253	pay-api-04	1 hora	4	92.95
254	pay-api-04	1 hora	5	94.65
255	pay-api-04	1 hora	6	96.36
256	pay-api-04	1 hora	7	98.06
257	pay-api-04	6 horas	0	49.24
258	pay-api-04	6 horas	1	51.19
259	pay-api-04	6 horas	2	53.14

	servidor	periodo	cpu	media_percent
260	pay-api-04	6 horas	3	55.09
261	pay-api-04	6 horas	4	57.04
262	pay-api-04	6 horas	5	58.99
263	pay-api-04	6 horas	6	60.94
264	pay-api-04	6 horas	7	62.89
265	pay-api-04	12 horas	0	43.02
266	pay-api-04	12 horas	1	44.99
267	pay-api-04	12 horas	2	46.97
268	pay-api-04	12 horas	3	48.94
269	pay-api-04	12 horas	4	50.92
270	pay-api-04	12 horas	5	52.89
271	pay-api-04	12 horas	6	54.87
272	pay-api-04	12 horas	7	56.84
273	pay-api-04	24 horas	0	39.01
274	pay-api-04	24 horas	1	41.00
275	pay-api-04	24 horas	2	42.99
276	pay-api-04	24 horas	3	44.97
277	pay-api-04	24 horas	4	46.96
278	pay-api-04	24 horas	5	48.95
279	pay-api-04	24 horas	6	50.93
280	pay-api-04	24 horas	7	52.92
281	pay-api-04	7 dias	0	36.47
282	pay-api-04	7 dias	1	38.47
283	pay-api-04	7 dias	2	40.47
284	pay-api-04	7 dias	3	42.47
285	pay-api-04	7 dias	4	44.47
286	pay-api-04	7 dias	5	46.46
287	pay-api-04	7 dias	6	48.46
288	pay-api-04	7 dias	7	50.46

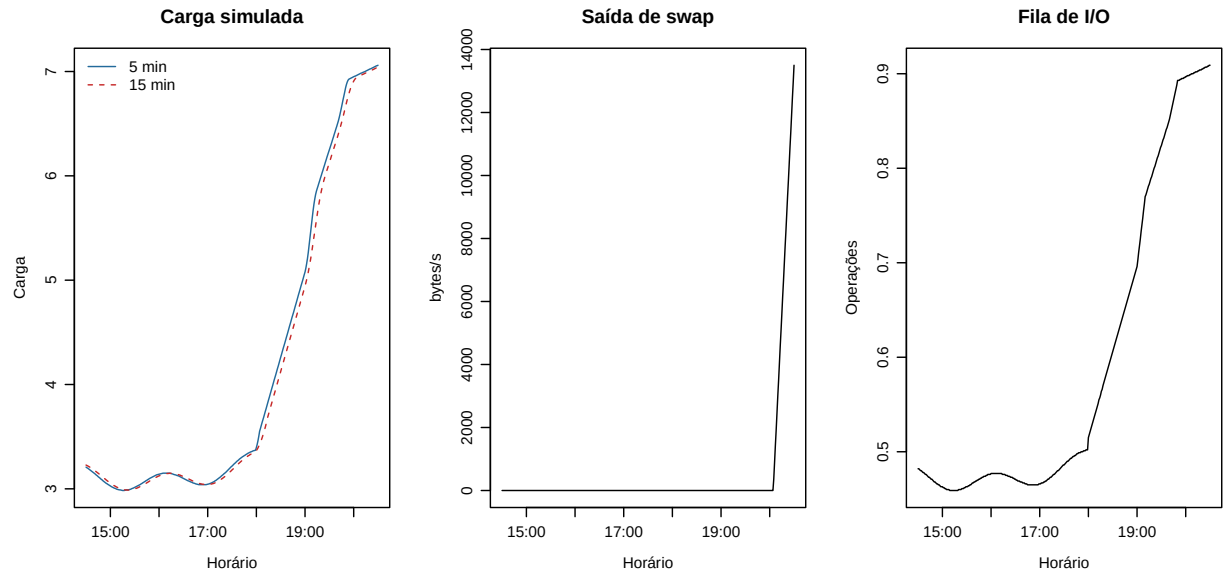
```
knitr::kable(dados_atuais[c("servidor", "ram_total", "ram_percent", "ram_available", "ram_used")], digi
```

	servidor	ram_total	ram_percent	ram_available	ram_used
10081	cart-api-02	16	64	5.76	10.24
20162	cart-api-07	16	81	3.04	12.96
30243	cart-api-09	16	57	6.88	9.12
40324	cart-api-12	16	63	5.92	10.08
50405	pay-db-02	32	79	6.72	25.28
60486	pay-api-04	16	83	2.72	13.28
70567	pay-api-06	16	62	6.08	9.92
80648	catalog-api-03	16	59	6.56	9.44
90729	catalog-api-05	64	66	21.76	42.24
100810	catalog-api-08	16	55	7.20	8.80

O gráfico por núcleo representa um instante; a tabela de médias representa o período e permite comparar os medidores do detalhe. Os dois usam a mesma distribuição exportada. O `catalog-api-05` tem 64 GB de RAM; seu percentual de ocupação é interpretado junto da quantidade disponível em GB. CPU e RAM crescem juntas neste exemplo porque o cenário foi montado assim. Isso não prova que uma causou o aumento da outra.

Carga, swap e I/O

```
dados_pay_api04 = historico_servidores[historico_servidores$servidor == "pay-api-04" & historico_servido
par(mfrow = c(1, 3), mar = c(5.5, 4.5, 3.5, 1), mgp = c(2.5, 0.8, 0))
plot(dados_pay_api04$instante, dados_pay_api04$load_avg_5, type = "l", col = "#236A9B", main = "Carga si
     xlab = "Horário", ylab = "Carga")
lines(dados_pay_api04$instante, dados_pay_api04$load_avg_15, col = "#C62828", lty = 2)
legend("topleft", c("5 min", "15 min"), col = c("#236A9B", "#C62828"), lty = c(1, 2), bty = "n")
plot(dados_pay_api04$instante, dados_pay_api04$swap_out_bytes_s, type = "l", main = "Saída de swap", xla
plot(dados_pay_api04$instante, dados_pay_api04$io_queue, type = "l", main = "Fila de I/O", xlab = "Horá
```




```
par(mfrow = c(1, 1))

knitr::kable(dados_atuais[c("servidor", "load_avg_5", "load_avg_15", "swap_percent",
  "swap_total_gb", "swap_used_gb", "swap_free_gb", "swap_in_bytes_s", "swap_out_bytes_s",
  "io_dispositivo", "io_read_ms", "io_write_ms", "io_queue", "io_read_iops", "io_write_iops")], digits =
```

	servidor	load_avg_5	load_avg_15	swap_percent	swap_total_gb	swap_used_gb	swap_free_gb	swap_in_bytes_s	swap_out_bytes_s	io_dispositivo	io_read_ms	io_write_ms	io_queue	io_read_iops	io_write_iops
10081	cart-api-02	4.238	4.227	10.0	4	0.400	3.600	0	0	nvme0n2	2.19	3.07	0.597	171	70
20162	cart-api-07	6.976	6.931	19.1	4	0.764	3.236	48000	4500	nvme0n2	2.93	4.00	0.901	276	116
30243	cart-api-09	3.432	3.447	10.0	4	0.400	3.600	0	0	nvme0n1	1.97	2.80	0.507	152	64
40324	cart-api-12	5.798	5.786	10.0	4	0.400	3.600	0	0	nvme0n2	2.61	3.60	0.770	243	100

	servidor	load_avg_5	load_avg_15	swap_pct	swap_total_gb	swap_used_gb	swap_free_gb	swap_in_bytes_s	swap_out_bytes_s	disco_pior	disco_read_ms	disco_write_ms	disco_queue	read_iops	write_iops
50405	pay-db-02	4.850	4.885	17.7	8	1.416	6.584	32000	0	nvme1n1	4.15	5.28	1.625	388	508
60486	pay-api-04	7.060	7.042	20.5	4	0.820	3.180	64000	13500	nvme0n2	2.95	4.03	0.909	271	112
70567	pay-api-06	4.310	4.298	10.0	4	0.400	3.600	0	0	nvme0n2	2.21	3.10	0.606	178	76
80648	catalog-api-03	4.020	4.017	10.0	4	0.400	3.600	0	0	nvme0n2	2.13	3.00	0.573	183	74
90729	catalog-api-05	3.806	3.835	10.0	8	0.800	7.200	0	0	nvme0n2	2.07	2.92	0.548	163	66
100810	catalog-api-08	3.212	3.241	10.0	4	0.400	3.600	0	0	nvme0n1	1.91	2.72	0.483	137	59

A carga de 5 e 15 minutos usa médias móveis simples da série simulada de 1 minuto. Essa é a regra do cenário, diferente do cálculo real do Linux. Swap e I/O também são valores simulados.

Uma carga alta pode envolver tarefas esperando CPU ou disco. Já a swap pode continuar ocupada depois de um período de maior uso. Por isso, olhamos esses sinais em conjunto. Para saber o impacto no serviço, também precisamos do tempo de resposta e dos erros da aplicação.

Volumes, espaço livre e inodes

```
knitr::kable(volumes_finais[c("servidor", "volume", "total_gb", "usado_gb", "livre_gb",
                              "ocupacao_percent", "inodes_usados_percent")], digits = 2)
```

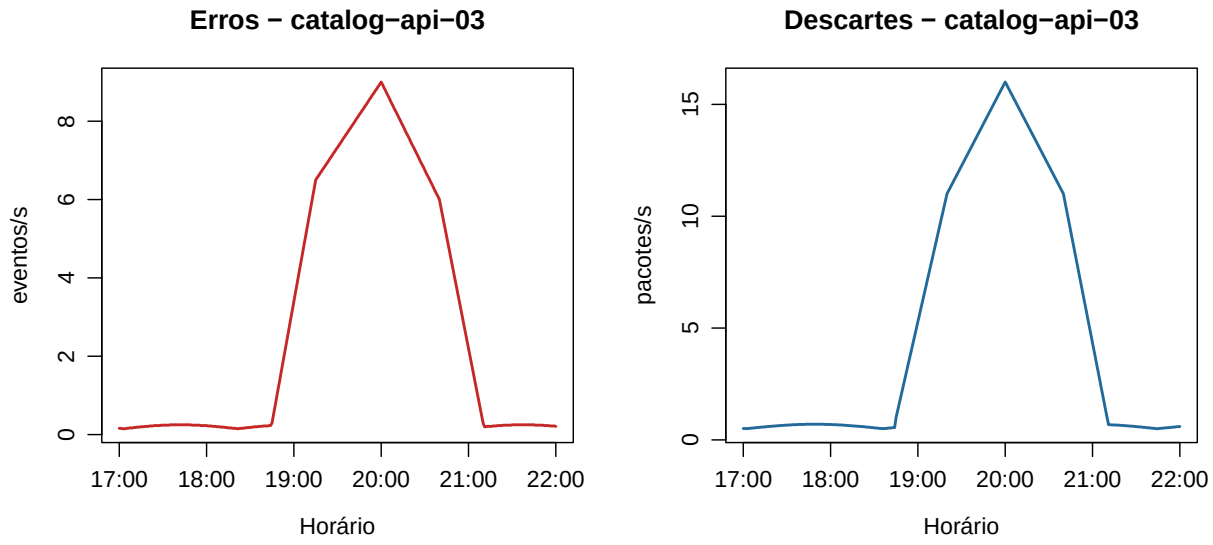
	servidor	volume	total_gb	usado_gb	livre_gb	ocupacao_percent	inodes_usados_percent
20161	cart-api-07	/dados	200	116.00	84.00	58.00	24.00
20162	cart-api-07	/	120	42.96	77.04	35.80	16.00
40323	cart-api-12	/dados	200	106.00	94.00	53.00	24.70
40324	cart-api-12	/	120	45.80	74.20	38.17	16.95
60485	cart-api-02	/dados	200	108.00	92.00	54.00	25.40
60486	cart-api-02	/	120	49.28	70.72	41.07	17.90
80647	cart-api-09	/dados	200	104.00	96.00	52.00	26.10
80648	cart-api-09	/	120	52.14	67.86	43.45	18.85
100809	pay-db-02	/dados	200	142.00	58.00	71.00	26.89
100810	pay-db-02	/	160	71.55	88.45	44.72	19.80
120971	pay-api-04	/dados	200	132.00	68.00	66.00	27.50
120972	pay-api-04	/	120	54.40	65.60	45.33	20.75
141133	pay-api-06	/dados	200	128.00	72.00	64.00	28.20
141134	pay-api-06	/	120	55.60	64.40	46.33	21.70
161295	catalog-api-03	/dados	200	126.00	74.00	63.00	28.90

	servidor	volume	total_gb	usado_gb	livre_gb	ocupacao_percent	inodes_usados_percent
161296	catalog-api-03	/	120	58.08	61.92	48.40	22.65
181457	catalog-api-05	/dados	500	385.00	115.00	77.00	30.23
181458	catalog-api-05	/	120	61.50	58.50	51.25	23.60
201619	catalog-api-08	/dados	200	98.00	102.00	49.00	30.30
201620	catalog-api-08	/	120	64.64	55.36	53.87	24.55

`/dados` e `/` são volumes diferentes e permanecem separados. O percentual de inodes indica a ocupação das estruturas usadas para registrar arquivos. A simulação não fornece as contagens absolutas de inodes; por isso não estimamos dias até seu esgotamento. O dispositivo de I/O é outra identificação, sem assumir que ele é o mesmo recurso que cada volume.

Rede no sábado e tráfego

```
dados_rede_sabado = historico_servidores[
  historico_servidores$servidor == "catalog-api-03" &
  historico_servidores$instante >= converter_horario("2026-10-17 17:00:00") &
  historico_servidores$instante <= converter_horario("2026-10-17 22:00:00"), ]
par(mfrow = c(1, 2), mar = c(5.5, 4.5, 3.5, 1), mgp = c(2.5, 0.8, 0))
plot(dados_rede_sabado$instante, dados_rede_sabado$net_errors, type = "l", col = "#C62828", lwd = 2,
  main = "Erros - catalog-api-03", xlab = "Horário", ylab = "eventos/s")
plot(dados_rede_sabado$instante, dados_rede_sabado$net_drops, type = "l", col = "#236A9B", lwd = 2,
  main = "Descartes - catalog-api-03", xlab = "Horário", ylab = "pacotes/s")
```



```
par(mfrow = c(1, 1))
knitr::kable(dados_atuais[c("servidor", "rede_interface", "rede_recv_mb_s", "rede_sent_mb_s",
  "rede_errin_s", "rede_errout_s", "rede_dropin_s", "rede_dropout_s")], digits = 3)
```

	servidor	rede_interface	rede_recv_mb_s	rede_sent_mb_s	rede_errin_s	rede_errout_s	rede_dropin_s	rede_dropout_s
10081	cart-api-02	eth0	5.48	2.148	0.143	0.047	0.448	0.242
20162	cart-api-07	eth0	8.81	3.406	0.165	0.055	0.429	0.231
30243	cart-api-09	eth0	4.49	1.774	0.188	0.062	0.332	0.178
40324	cart-api-12	eth0	7.37	2.862	0.173	0.057	0.384	0.206
50405	pay-db-02	eth0	4.90	1.770	0.112	0.038	0.455	0.245
60486	pay-api-04	eth0	7.00	2.490	0.188	0.062	0.370	0.200
70567	pay-api-06	eth0	4.41	1.602	0.143	0.047	0.436	0.234
80648	catalog-api-03	eth0	3.05	1.830	0.173	0.057	0.410	0.220
90729	catalog-api-05	eth0	2.90	1.740	0.165	0.055	0.403	0.217
100810	catalog-api-08	eth0	2.50	1.500	0.150	0.050	0.442	0.238

O episódio de rede termina antes do instante final. Erros e descartes são apresentados separadamente e identificados por interface e direção. Tráfego está em MB/s decimais; erros e descartes têm unidades próprias. Esses dados não são percentuais de rede nem contagens de pedidos perdidos. Estado da interface e taxas de pacotes fazem parte do catálogo, mas não estão presentes nesta base.

Processos, containers e OOM

```
knitr::kable(processos_atuais[processos_atuais$servidor == "pay-api-04",
  c("servidor", "pid", "nome", "estado", "cpu_percent", "ram_gb")], digits = 3)
```

	servidor	pid	nome	estado	cpu_percent	ram_gb
16	pay-api-04	2405	node · pay-api-04	Executando	243.2	3.851
17	pay-api-04	2417	pay-worker	Executando	91.2	1.594
18	pay-api-04	2463	coretrace-agent	Em espera	3.8	0.066

```
knitr::kable(containers_atuais[containers_atuais$servidor == "pay-api-04",
  c("servidor", "id", "nome", "estado", "cpu_percent", "ram_gb", "leitura_bytes_s", "escrita_bytes_s")]
```

	servidor	id	nome	estado	cpu_percent	ram_gb	leitura_bytes_s	escrita_bytes_s
16	pay-api-04	demo-6-1	pay-api-04-app	Executando	288.80	4.515	6467200	2878200
17	pay-api-04	demo-6-2	pay-api-04-worker	Executando	114.00	2.125	2707200	1193400

	servidor	id	nome	estado	cpu_percent	ram_gb	leitura_bytes_s	escrita_bytes_s
18	pay-api-04	demo-6-3	pay-api-04-agent	Executando	4.56	0.106	225600	70200

```
knitr::kable(cadastro_servidores[c("servidor", "perfil", "docker")])
```

servidor	perfil	docker
cart-api-07	Carrinho	TRUE
cart-api-12	Carrinho	TRUE
cart-api-02	Carrinho	TRUE
cart-api-09	Carrinho	TRUE
pay-db-02	Financeiro	TRUE
pay-api-04	Financeiro	TRUE
pay-api-06	Financeiro	TRUE
catalog-api-03	Catálogo	FALSE
catalog-api-05	Catálogo	FALSE
catalog-api-08	Catálogo	FALSE

```
# Quantidade de consumidores no instante final.
knitr::kable(data.frame(tipo = c("Processos", "Containers"),
  quantidade = c(nrow(processos_atuais), nrow(containers_atuais))))
```

tipo	quantidade
Processos	30
Containers	21

```
oom_periodos = data.frame()
for (periodo_atual in names(periodos)) {
  dados_periodo = selecionar_periodo(periodo_atual)
  if (any(is.na(dados_periodo$ram_oom_kills))) stop("Há valores ausentes na coluna de OOM.")
  oom_periodos = rbind(oom_periodos, data.frame(periodo = periodo_atual,
    ocorrencias_oom = sum(dados_periodo$ram_oom_kills)))
}
knitr::kable(oom_periodos)
```

periodo	ocorrencias_oom
30 minutos	0
1 hora	0
6 horas	0
12 horas	0
24 horas	0
7 dias	0

Os processos e containers representam os consumidores às 20h30. O uso de CPU segue a convenção de 100% por CPU lógica, podendo passar de 100%. Um processo pode estar dentro de um container; somar os dois níveis pode duplicar consumo. No perfil Catálogo, Docker não é monitorado.

Não há ocorrências de OOM na semana simulada. Esse resultado descreve o cenário e não representa uma medição do contador real do Linux.

Previsão de capacidade: geral e detalhe

A previsão utiliza duas janelas de histórico. Na geral, a previsão usa apenas os pontos horários do período selecionado e exige pelo menos três pontos e 24 horas. No detalhe, usa os pontos horários da semana inteira. Nos dois casos, só há previsão quando o crescimento é suficiente; nesta simulação, o mínimo é 0,25 ponto percentual por dia.

O crescimento diário é calculado pela diferença entre a primeira e a última ocupação da janela, dividida pela duração em dias. A geral arredonda o prazo para cima; o detalhe apresenta uma casa decimal e não oferece previsão de um volume já no limite crítico.

```
# Tamanho das margens e dos rótulos do gráfico.
par(mar = c(5.5, 4.5, 3.5, 1.5), mgp = c(2.5, 0.8, 0), cex = 0.9)
previsoes_geral = data.frame()
previsoes_detalhe = data.frame()
for (indice_servidor in seq_len(nrow(cadastro_servidores))) {
  servidor_atual = cadastro_servidores[indice_servidor, ]
  dados_servidor = historico_servidores[historico_servidores$mac == servidor_atual$mac, ]
  # A dash amostra o histórico a cada 60 minutos a partir do início da semana.
  coletas_por_hora = dados_servidor[seq(1, nrow(dados_servidor), by = 60), ]
  limite_disco = limites_alerta[limites_alerta$perfil == servidor_atual$perfil & limites_alerta$metrica
  ocupacao_atual = tail(dados_servidor$disk_percent, 1)
  crescimento_diario_semana = (tail(coletas_por_hora$disk_percent, 1) - coletas_por_hora$disk_percent[1
  prazo_detalhe = NA_real_
  if (crescimento_diario_semana >= 0.25 && ocupacao_atual < limite_disco$critico) {
    prazo_detalhe = round((limite_disco$critico - ocupacao_atual) / crescimento_diario_semana, 1)
  }
  previsoes_detalhe = rbind(previsoes_detalhe, data.frame(servidor = servidor_atual$servidor,
    perfil = servidor_atual$perfil, volume = "/dados", periodo_calculo = "7 dias",
    ocupacao_percent = ocupacao_atual, crescimento_pp_dia = round(crescimento_diario_semana, 3),
    limite_critico = limite_disco$critico, dias = prazo_detalhe))

  for (periodo_atual in names(periodos)) {
    coletas_horarias_periodo = coletas_por_hora[coletas_por_hora$instante >= horario_final - periodos[pe
    duracao_dias = as.numeric(difftime(tail(coletas_horarias_periodo$instante, 1), coletas_horarias_peri
    crescimento_diario = NA_real_
    prazo_previsto = NA_real_
    situacao_previsao = "Dados insuficientes"
    if (nrow(coletas_horarias_periodo) >= 3 && duracao_dias >= 1) {
      crescimento_diario = (tail(coletas_horarias_periodo$disk_percent, 1) - coletas_horarias_periodo$di
      situacao_previsao = "Sem tendência"
      if (crescimento_diario > 0 && crescimento_diario >= 0.25) {
        prazo_bruto = (limite_disco$critico - tail(coletas_horarias_periodo$disk_percent, 1)) / crescime
        # Limita erros de precisão antes de arredondar o prazo para cima.
        prazo_previsto = max(0, ceiling(round(prazo_bruto, 8)))
        situacao_previsao = "Com previsão"
      }
    }
    previsoes_geral = rbind(previsoes_geral, data.frame(servidor = servidor_atual$servidor,
      perfil = servidor_atual$perfil, periodo = periodo_atual, pontos = nrow(coletas_horarias_periodo),
      dias = prazo_previsto, resultado = situacao_previsao))
  }
}

# O card geral escolhe o menor prazo disponível entre os servidores do perfil.
```

```
cards_previsao = data.frame()
for (perfil_atual in perfis_analisados) {
  for (periodo_atual in names(periodos)) {
    previsoes_periodo = previsoes_geral[previsoes_geral$periodo == periodo_atual, ]
    if (perfil_atual != "Todos") previsoes_periodo = previsoes_periodo[previsoes_periodo$perfil == perfi
    previsoes_disponiveis = previsoes_periodo[!is.na(previsoes_periodo$dias), ]
    servidor_previsao = ""
    dias_previstos = NA_real_
    situacao_previsao = if (periodos[periodo_atual] < 1440) "Dados insuficientes" else "Sem tendência"
    if (nrow(previsoes_disponiveis) > 0) {
      menor_previsao = previsoes_disponiveis[which.min(previsoes_disponiveis$dias), ]
      servidor_previsao = menor_previsao$servidor
      dias_previstos = menor_previsao$dias
      situacao_previsao = "Com previsão"
    }
    cards_previsao = rbind(cards_previsao, data.frame(perfil = perfil_atual,
      periodo = periodo_atual, servidor = servidor_previsao, dias = dias_previstos, resultado = situaca
  }
}
knitr::kable(previsoes_geral[previsoes_geral$servidor == "catalog-api-05", ], digits = 3)
```

	servidor	perfil	periodo	pontos	crescimento_pp_dia	dias	resultado
49	catalog-api-05	Catálogo	30 minutos	1	NA	NA	Dados insuficientes
50	catalog-api-05	Catálogo	1 hora	2	NA	NA	Dados insuficientes
51	catalog-api-05	Catálogo	6 horas	7	NA	NA	Dados insuficientes
52	catalog-api-05	Catálogo	12 horas	13	NA	NA	Dados insuficientes
53	catalog-api-05	Catálogo	24 horas	25	1.5	12	Com previsão
54	catalog-api-05	Catálogo	7 dias	169	1.5	12	Com previsão

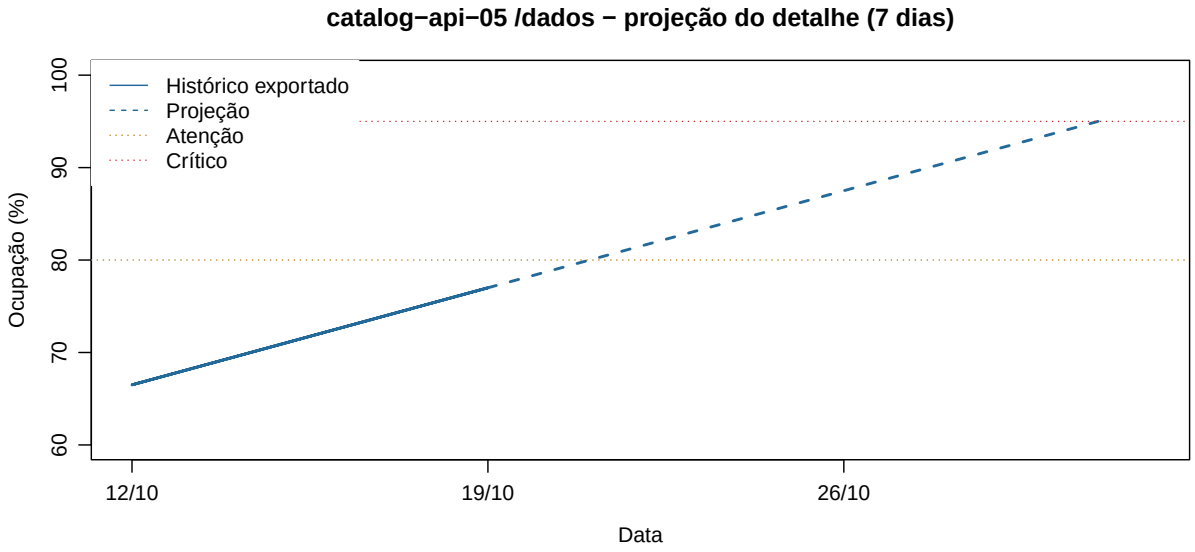
```
knitr::kable(previsoes_detalhe, digits = 3)
```

servidor	perfil	volume	periodo_calculo	ocupacao_percent	crescimento_pp_dia	limite_critico	dias
cart-api-07	Carrinho	/dados	7 dias	58	0.04	95	NA
cart-api-12	Carrinho	/dados	7 dias	53	0.04	95	NA
cart-api-02	Carrinho	/dados	7 dias	54	0.04	95	NA
cart-api-09	Carrinho	/dados	7 dias	52	0.04	95	NA
pay-db-02	Financeiro	/dados	7 dias	71	0.04	95	NA
pay-api-04	Financeiro	/dados	7 dias	66	0.04	95	NA
pay-api-06	Financeiro	/dados	7 dias	64	0.04	95	NA
catalog-api-03	Catálogo	/dados	7 dias	63	0.04	95	NA
catalog-api-05	Catálogo	/dados	7 dias	77	1.50	95	12
catalog-api-08	Catálogo	/dados	7 dias	49	0.04	95	NA

```
knitr::kable(cards_previsao)
```

perfil	periodo	servidor	dias	resultado
Todos	30 minutos		NA	Dados insuficientes
Todos	1 hora		NA	Dados insuficientes
Todos	6 horas		NA	Dados insuficientes
Todos	12 horas		NA	Dados insuficientes
Todos	24 horas	catalog-api-05	12	Com previsão
Todos	7 dias	catalog-api-05	12	Com previsão
Carrinho	30 minutos		NA	Dados insuficientes
Carrinho	1 hora		NA	Dados insuficientes
Carrinho	6 horas		NA	Dados insuficientes
Carrinho	12 horas		NA	Dados insuficientes
Carrinho	24 horas		NA	Sem tendência
Carrinho	7 dias		NA	Sem tendência
Financeiro	30 minutos		NA	Dados insuficientes
Financeiro	1 hora		NA	Dados insuficientes
Financeiro	6 horas		NA	Dados insuficientes
Financeiro	12 horas		NA	Dados insuficientes
Financeiro	24 horas		NA	Sem tendência
Financeiro	7 dias		NA	Sem tendência
Catálogo	30 minutos		NA	Dados insuficientes
Catálogo	1 hora		NA	Dados insuficientes
Catálogo	6 horas		NA	Dados insuficientes
Catálogo	12 horas		NA	Dados insuficientes
Catálogo	24 horas	catalog-api-05	12	Com previsão
Catálogo	7 dias	catalog-api-05	12	Com previsão

```
previsao_catalogo = previsoes_detalhe[previsoes_detalhe$servidor == "catalog-api-05", ]
dados_servidor = historico_servidores[historico_servidores$servidor == "catalog-api-05", ]
plot(dados_servidor$instante, dados_servidor$disk_percent, type = "l", lwd = 2, col = "#236A9B",
  xlim = c(inicio_semana, horario_final + 13 * 86400), ylim = c(60, 100),
  main = "catalog-api-05 /dados - projeção do detalhe (7 dias)",
  xlab = "Data", ylab = "Ocupação (%)", xaxt = "n")
marcas_data = seq(inicio_semana, horario_final + 13 * 86400, by = "7 days")
axis(1, at = marcas_data, labels = format(marcas_data, "%d/%m", tz = "America/Sao_Paulo"))
lines(c(horario_final, horario_final + previsao_catalogo$dias * 86400), c(previsao_catalogo$ocupacao_pe
  lty = 2, lwd = 2, col = "#236A9B")
abline(h = c(80, previsao_catalogo$limite_critico), col = c("#B77900", "#C62828"), lty = 3)
legend("topleft", c("Histórico exportado", "Projeção", "Atenção", "Crítico"),
  col = c("#236A9B", "#236A9B", "#B77900", "#C62828"), lty = c(1, 2, 3, 3),
  bty = "o", bg = "white", box.col = "white")
```



Com a semana inteira, `(95 - 77) / 1,5 = 12 dias` até atingir 95%. O crescimento é de 1,5 ponto percentual por dia, equivalente a 7,5 GB/dia num volume de 500 GB. Com operador `>`, estar exatamente em 95% ainda não é ultrapassar o limite crítico. A ocupação de 77% também está abaixo da atenção de 80%: a previsão apoia planejamento e não é um alerta ativo.

Os 12 dias não são repetidos automaticamente nos filtros curtos. Neles, a geral informa dados insuficientes. No detalhe, a previsão continua semanal. A projeção depende de crescimento constante definido no cenário; limpeza, expansão e mudança da demanda podem invalidá-la em uma operação real.

Justificativa das métricas essenciais

Usamos a metodologia USE como referência para pensar em três perguntas: quanto do recurso está em uso, se há trabalho esperando por ele e se há erros. As métricas do CoreTrace ajudam nessas perguntas, mas não cobrem todos os aspectos da metodologia. A escolha das métricas obrigatórias faz parte da proposta do produto.

Grupo do catálogo	Pergunta apoiada	O que foi analisado e qual é o limite
CPU total e por CPU lógica	Quanto processamento é usado e onde se concentra?	Classificação total, valores por núcleo e médias por período; não comprovam lentidão da aplicação
RAM percentual, disponível e total	Qual a ocupação e a margem em GB?	Capacidades e valores exportados, incluindo 64 GB no catalog-api-05; a relação entre usada e disponível segue a convenção da simulação
OOM	Houve encerramento por falta de memória?	Valores informados e iguais a zero; não testa o contador real do Linux nem reinício
Disco percentual e livre	Quanto está ocupado e quanto resta?	Histórico e valores atuais por volume, com tendência de capacidade; não mede desempenho por si só

Grupo do catálogo	Pergunta apoiada	O que foi analisado e qual é o limite
Inodes	Ainda há capacidade de registrar arquivos?	Percentuais exportados por volume; não há contagens absolutas nem previsão de esgotamento
Tempo estimado e fila de I/O	Há atividade acompanhada de espera?	Valores sintéticos por dispositivo e operação; não são tempo de resposta da aplicação nem validação de limites reais
Swap percentual, entrada e saída	Há ocupação e movimentação?	Valores do gerador, representam a ocupação e movimentação definidas para o cenário
Carga de 5 e 15 minutos	A demanda persiste?	Médias móveis da série simulada de 1 minuto; não reproduzem o cálculo real do Linux
Rede	Qual o tráfego e quais ocorrências há por interface?	Tráfego, erros e descartes exportados; estado da interface e taxas de pacotes não foram fornecidos nos CSVs
Processos e containers	Quem acompanha o consumo do servidor?	Listas atuais de consumidores, com CPU e RAM; sem vínculo comprovado entre processo e container nem histórico desses consumidores

A necessidade de cada métrica depende do ambiente e das perguntas de investigação. Modos de CPU, frequência, interrupções, trocas de contexto e demais campos não exportados continuam como itens do catálogo, sem análise numérica neste estudo. O produto deve aceitar não aplicabilidade e a seleção pode ser revista conforme a necessidade.

Como a dashboard apoia as decisões

Elemento	Pergunta	Decisão apoiada	Limitação
Cards de servidores no período	Quais servidores chegaram a atenção ou crítico?	Priorizar a investigação do intervalo	A maior gravidade histórica não é necessariamente o estado atual
Alertas do período	Quantos episódios por componente ocorreram na janela?	Localizar ocorrências, inclusive recuperadas	Não equivale a servidores, condições atuais ou tickets
Alertas ativos e valores atuais	O que continua fora do normal às 20h30?	Investigar os componentes ainda afetados	Não garante disponibilidade da aplicação
Previsão de capacidade	Em quanto tempo o volume chegaria ao limite?	Avaliar limpeza ou expansão	Geral e detalhe usam janelas diferentes; depende da tendência
Evolução temporal	Como as contagens mudaram?	Investigar início, escalada e recuperação	Resolução de um minuto perde eventos menores

Elemento	Pergunta	Decisão apoiada	Limitação
Barras por recurso no período	Onde ocorreram condições de maior gravidade?	Direcionar a investigação	O mesmo servidor pode aparecer em mais de um recurso
Detalhes e consumidores	Que outros sinais acompanham o episódio?	Formular e conferir hipóteses	Dois sinais variarem juntos não explica, sozinho, a causa

No cenário de delivery, os perfis ajudam a separar carrinho, pagamentos e catálogo. O histórico também permite comparar almoço, jantar e campanha. Um perfil não é automaticamente mais importante que outro: a prioridade depende da gravidade do problema e do impacto no serviço.

CPU, RAM, disco e rede permitem acompanhar os problemas definidos no cenário. Carga, swap, I/O, processos e containers ajudam a investigar. Isso não cobre todo problema de uma operação de delivery. Dependendo da necessidade, também é preciso acompanhar tempo de resposta, erros da aplicação, pedidos, pagamentos, banco de dados e filas.

Conclusão

Na semana simulada, os servidores apresentam episódios de aumento de CPU, memória e ocorrências de rede. Parte deles se recupera antes do instante final. Às 20h30, quatro servidores continuam afetados, com dois críticos, dois em atenção e seis condições de alerta ativas.

A evolução temporal e os resumos por período ajudam a distinguir episódios recuperados dos que continuam ativos. O detalhamento por recurso, núcleo, volume e consumidor acrescenta contexto à investigação. No armazenamento, a ocupação de 77% do volume de 500 GB do `catalog-api-05` ainda está abaixo do limite de atenção, mas o crescimento de 1,5 ponto percentual por dia indica 12 dias até atingir 95%, considerando a semana inteira.

Com essas métricas, conseguimos analisar os episódios da simulação. Para tirar conclusões sobre uma operação real e saber se houve impacto em pedidos ou pagamentos, seriam necessários dados reais dos servidores e da aplicação.

Referências e origem dos dados

- Cenário narrativo: `cenario-fornecido.txt`.
- Dados: arquivos CSV da pasta indicada em `pasta_csv`, gerados pela simulação do CoreTrace.
- Fórmulas da simulação: `website/public/scripts/cenario/cenario-coretrace-7dias.js` e `detalhamento-servidor.js`.
- Regras da geral: `dashboardDados.js` e `dashboardSimulacao.js`; os limites exportados são referência do código, não consulta ao banco.
- Brendan Gregg — The USE Method: utilização, saturação e erros; não define as obrigatoriedades do produto.
- psutil — memória: distinções entre os campos de memória de uma coleta real.

Exportação dos resultados

As tabelas calculadas são salvas em uma pasta separada dos arquivos de entrada.

```r
dir.create("resultados-analise", showWarnings = FALSE)
salvar_resultado = function(tabela, nome) {
  write.csv(tabela, file.path("resultados-analise", nome), row.names = FALSE,
            na = "", fileEncoding = "UTF-8")
}
salvar_resultado(resumo_dados, "resumo-dados.csv")
salvar_resultado(incidentes_jantar, "incidentes-jantar.csv")
salvar_resultado(cards_periodo, "cards-periodos-perfis.csv")
salvar_resultado(barras_periodo, "barras-periodos-perfis.csv")
salvar_resultado(evolucao, "evolucao.csv")
salvar_resultado(incidentes, "incidentes.csv")
salvar_resultado(alertas_ativos, "alertas-ativos.csv")
salvar_resultado(indicadores_finais, "indicadores-finais.csv")
salvar_resultado(dados_atuais[c("servidor", "mac", "perfil", "cpu_percent", "ram_percent", "ram_total",
  "ram_used", "ram_available", "situacao")], "servidores-finais.csv")
salvar_resultado(medias_nucleos, "medias-nucleos-periodos.csv")
salvar_resultado(estatisticas_servidores, "estatisticas.csv")
salvar_resultado(previsoes_geral, "previsoes-geral.csv")
salvar_resultado(previsoes_detalhe, "previsoes-detalhe.csv")
salvar_resultado(cards_previsao, "cards-previsao.csv")
salvar_resultado(oom_periodos, "oom-periodos.csv")
```